



Universidad Técnica Estatal de Quevedo

Facultad de ciencias de la computación

Ingeniería en software

Asignatura:

Ingeniería de requerimientos

Tema:

Practica experimental III

Integrantes:

Chavarría Cuenca Tahiny Mel

Muñoz Quiñonez Yeranick Esther

Valarezo Flores Steven Alexander

Valdiviezo Yun On Adamo Eduardo

Docente:

PhD. Guerrero Ulloa Gleiston Ciceron

Fecha:

07/07/2026

REGULAR - 2026-2027 PPA

Contenido

1.	Introducción.....	5
2.	Fundamento teórico	6
2.1.	Marco normativo aplicado a la especificación de requisitos	6
2.1.1.	Evolución de los estándares para la especificación de requisitos	6
2.1.2.	Organismos de normalización: IEEE, ISO e IEC	7
2.1.3.	Estándar IEEE/ISO/IEC 29148	7
2.1.4.	Importancia del uso de estándares.....	8
2.2.	Documentación de requisitos	8
2.2.1.	Objetivos	8
2.2.2.	Ciclo de vida de los requisitos.....	8
2.2.3.	Requisitos funcionales	9
2.2.4.	Requisitos no funcionales.....	9
2.2.5.	Casos de uso	10
2.2.6.	Historias de usuario.....	10
2.3.	Documento ERS/SRS según IEEE/ISO/IEC 29148	10
2.3.1.	Objetivos del documento ERS/SRS	11
2.3.2.	Beneficios del documento ERS/SRS	11
2.3.3.	Estructura recomendada según IEEE/ISO/IEC 29148	11
2.3.4.	Aplicación del documento ERS/SRS en UteqMarket	12
2.4.	Modelos de datos (Perspectiva de datos).....	13
2.4.1.	Modelado conceptual.....	13
2.4.2.	Modelo entidad relación (ER)	14
2.4.3.	Entidades.....	14
2.4.4.	Relaciones	14
2.4.5.	Cardinalidades.....	14
2.4.6.	Notación Crow's Foot	15
2.4.7.	Normalización	15
2.4.8.	Llaves	16
2.4.9.	Diagrama de clases.....	17
2.5.	Modelos funcionales	17
2.5.1.	Perspectiva funcional.....	17

2.5.2.	Flujo de datos	18
2.5.3.	Diagrama de Flujo de Datos (DFD)	18
2.5.4.	Diagrama de Flujo de Datos Nivel 0	19
2.5.5.	Diagrama de Flujo de Datos Nivel 1	19
2.5.6.	DFD Esencial	19
2.5.7.	Diagrama de Actividades UML	19
2.6.	Modelos de comportamiento.....	19
2.6.1.	Máquina de estados	20
2.6.2.	Estados y eventos.....	20
2.6.3.	Diagrama de secuencia	21
2.7.	Interrelaciones entre las tres perspectivas.....	21
2.7.1.	Integración de los modelos	21
3.	Desarrollo de la practica	22
3.1.	Revisión del SRS	22
3.2.	Defectos encontrados	23
3.3.	Tabla completa de atributos	25
3.3.1.	Requisitos funcionales	25
3.3.2.	Requisitos no funcionales.....	29
3.4.	Modelo de datos	31
3.4.1.	Identificación de entidades y relaciones.....	31
3.4.2.	Diagrama ER.....	35
3.4.3.	Verificación de 3FN	36
3.4.4.	Diagrama de clases.....	36
3.5.	Modelo funcional.....	37
3.5.1.	DFD nivel 0	37
3.5.2.	DFD nivel 1	38
3.5.3.	DFD esencial.....	40
3.5.4.	Diagrama de actividades	42
3.6.	Modelos de comportamiento	43
3.6.1.	Selección del caso	43
3.6.2.	Diagrama de estados	44
3.6.3.	Diagrama de secuencia.....	45

3.7.	Matriz de trazabilidad	46
3.7.1.	Matriz RF x Modelos	46
3.7.2.	Detección de GAPs	56
3.7.3.	Identificación de gold plating.....	56
3.7.4.	Verificaciones cruzadas entre modelos.....	56
3.8.	Tabla comparativa de modelos	58
4.	Conclusiones.....	59
5.	Bibliografía	60

1. Introducción

La presente práctica experimental tiene como finalidad aplicar los conocimientos adquiridos en la asignatura de Ingeniería de Requerimientos mediante la elaboración y análisis de los principales modelos utilizados en la especificación de requisitos de software [1]. Para ello, se emplea el estándar IEEE/ISO/IEC 29148:2018, el cual proporciona las directrices para la documentación de requisitos de manera clara, completa, consistente y verificable, permitiendo mejorar la calidad del proceso de desarrollo de software [2].

Como caso de estudio se utilizará el proyecto UteqMarket, una aplicación web orientada a la comunidad de la Universidad Técnica Estatal de Quevedo (UTEQ), cuyo propósito es ofrecer un espacio seguro para la compra, venta e intercambio de productos y servicios entre estudiantes, docentes y personal administrativo mediante el uso exclusivo de correos institucionales. Este proyecto permitirá aplicar de forma práctica los conceptos de ingeniería de requerimientos y modelado de software vistos durante el curso [3].

Durante el desarrollo de la práctica se elaborarán diferentes modelos UML y modelos de análisis que representan distintas perspectivas del sistema. En la perspectiva de datos se desarrollarán el Diagrama Entidad–Relación (ER) y el Diagrama de Clases UML [4], con el fin de representar la estructura de la información y las relaciones existentes entre las entidades del sistema [5]. En la perspectiva funcional se construirán el Diagrama de Flujo de Datos (DFD) y el Diagrama de Actividades UML, los cuales describen los procesos del sistema y el flujo de información. Finalmente, en la perspectiva de comportamiento se desarrollarán el Diagrama de Estados y el Diagrama de Secuencia UML, permitiendo representar la evolución de los objetos y la interacción entre los diferentes elementos del sistema [6]

Asimismo, se realizará la revisión y refinamiento del documento de especificación de requisitos (SRS), identificando posibles inconsistencias y aplicando criterios de calidad establecidos por el estándar IEEE/ISO/IEC 29148:2018 [7]. Además, se elaborará una matriz de trazabilidad y una comparación entre los diferentes modelos desarrollados, con el propósito de evidenciar la relación existente entre los requisitos y las distintas perspectivas de análisis del sistema.

2. Fundamento teórico

2.1. Marco normativo aplicado a la especificación de requisitos

La Ingeniería de Requerimientos constituye una de las disciplinas fundamentales dentro de la Ingeniería de Software, ya que se encarga de identificar, analizar, documentar, validar y gestionar las necesidades y expectativas de los interesados respecto al sistema que se pretende desarrollar [8]. Su propósito principal es garantizar que el software responda a los objetivos del negocio y satisfaga las necesidades reales de los usuarios, reduciendo errores durante las etapas posteriores del desarrollo [2].

Una especificación de requisitos correctamente elaborada permite establecer una comunicación clara entre clientes, desarrolladores y demás participantes del proyecto, evitando ambigüedades y facilitando la planificación, el diseño, la implementación y las pruebas del software [3]. Diversos estudios han demostrado que una gran parte de los problemas presentes en proyectos informáticos tienen su origen en requisitos incompletos, ambiguos o incorrectamente documentados, lo que incrementa significativamente los costos de mantenimiento y corrección [1].

Por esta razón, la Ingeniería de Requerimientos no se limita únicamente a recopilar información sobre las necesidades del cliente, sino que también incorpora actividades de análisis, priorización, validación, negociación y trazabilidad de los requisitos durante todo el ciclo de vida del software [7].

2.1.1. Evolución de los estándares para la especificación de requisitos

A medida que la Ingeniería de Software evolucionó, surgió la necesidad de establecer normas internacionales que permitieran documentar los requisitos de manera uniforme y comprensible. Inicialmente muchas organizaciones utilizaban formatos propios para elaborar documentos de requisitos, lo que ocasionaba diferencias en la interpretación de la información y dificultades para mantener proyectos de gran escala [9].

Uno de los primeros estándares ampliamente aceptados fue el IEEE Std 830, publicado con el propósito de proporcionar una estructura para la elaboración de documentos de Especificación de Requisitos de Software (Software Requirements Specification, SRS). Durante varios años este estándar fue utilizado como referencia en proyectos académicos e industriales debido a la claridad de su estructura documental [10].

Sin embargo, el crecimiento de sistemas cada vez más complejos y la aparición de nuevas metodologías de desarrollo hicieron necesaria la creación de un estándar más amplio que integrara las mejores prácticas internacionales. Como resultado surgió el estándar ISO/IEC/IEEE 29148, el cual reemplaza al IEEE 830 e incorpora nuevas recomendaciones relacionadas con la calidad de los requisitos, la trazabilidad y la gestión

de diferentes tipos de especificaciones dentro del proceso de Ingeniería de Sistemas y Software [11].

2.1.2. Organismos de normalización: IEEE, ISO e IEC

El Institute of Electrical and Electronics Engineers (IEEE) es una organización internacional dedicada al desarrollo de estándares técnicos en áreas como informática, telecomunicaciones, electrónica e ingeniería de software. Sus normas son ampliamente utilizadas por la industria y las instituciones académicas debido a su reconocimiento internacional y constante actualización [12].

La International Organization for Standardization (ISO) es un organismo independiente encargado de desarrollar normas internacionales que promueven la calidad, la interoperabilidad y la mejora continua en diferentes sectores productivos y tecnológicos [13].

Por su parte, la International Electrotechnical Commission (IEC) desarrolla estándares relacionados con tecnologías eléctricas y electrónicas. En el ámbito del software trabaja conjuntamente con ISO para definir normas aplicables a los procesos de desarrollo, calidad y documentación de sistemas [14].

La colaboración entre estas tres organizaciones dio origen a estándares conjuntos ampliamente utilizados en Ingeniería de Software, entre ellos el ISO/IEC/IEEE 29148, considerado actualmente la principal referencia para la especificación de requisitos.

2.1.3. Estándar IEEE/ISO/IEC 29148

El estándar ISO/IEC/IEEE 29148:2018 establece las mejores prácticas para la Ingeniería de Requerimientos y define los procesos necesarios para identificar, documentar, validar y administrar requisitos durante todo el ciclo de vida de un sistema [2].

A diferencia de estándares anteriores, el IEEE/ISO/IEC 29148 no se limita únicamente a proporcionar un formato para redactar un documento SRS, sino que también describe las características que deben cumplir los requisitos para ser considerados de calidad, así como las relaciones existentes entre los diferentes tipos de especificaciones utilizados durante el desarrollo de sistemas [1].

Asimismo, el estándar incorpora recomendaciones para garantizar la consistencia entre requisitos funcionales, requisitos no funcionales, modelos de análisis y procesos de verificación, favoreciendo la trazabilidad y reduciendo la posibilidad de inconsistencias durante el desarrollo del software[3].

En esta práctica experimental se adopta dicho estándar como marco de referencia para la elaboración y refinamiento de los requisitos del proyecto UTeqMarket, permitiendo construir modelos de datos, funcionales y de comportamiento alineados con las recomendaciones internacionales [7].

2.1.4. Importancia del uso de estándares

La utilización de estándares internacionales durante la especificación de requisitos ofrece múltiples beneficios tanto en proyectos académicos como profesionales. En primer lugar, proporciona una estructura uniforme para documentar la información, facilitando la comunicación entre clientes, analistas, desarrolladores y demás interesados [1].

Además, la aplicación de estándares contribuye a disminuir errores de interpretación, mejorar la calidad de la documentación y facilitar actividades posteriores como el diseño, la implementación, las pruebas y el mantenimiento del sistema.

Otro aspecto importante es la trazabilidad, ya que permite relacionar cada requisito con los modelos desarrollados y con las funcionalidades implementadas, facilitando el control de cambios y la validación del software [3].

2.2. Documentación de requisitos

La documentación de requisitos es el proceso mediante el cual se identifican, organizan y describen las necesidades que debe cumplir un sistema de software. Este documento sirve como guía para el desarrollo del proyecto, ya que permite definir qué funcionalidades debe tener el sistema, cómo debe comportarse y qué condiciones debe cumplir para satisfacer a los usuarios [15].

En el caso de UTeqMarket, la documentación de requisitos permite establecer claramente las funciones principales de la plataforma, como la publicación de productos, la búsqueda de artículos, la gestión de usuarios y la comunicación entre compradores y vendedores.

2.2.1. Objetivos

- Describir de manera clara, completa y consistente las necesidades y expectativas de los interesados respecto al sistema [16].
- Establecer una base común de comunicación entre clientes, usuarios, analistas, desarrolladores y demás involucrados en el proyecto [16].
- Servir como fundamento para las etapas de diseño, implementación, pruebas y mantenimiento del software [16].
- Facilitar la gestión y trazabilidad de los requisitos, permitiendo controlar cambios y verificar su cumplimiento durante el ciclo de vida del proyecto [16].
- Definir criterios objetivos de validación y verificación, con el fin de comprobar que el sistema desarrollado satisface los requisitos especificados [16].

2.2.2. Ciclo de vida de los requisitos

El ciclo de vida de los requisitos comprende un conjunto de actividades que permiten definir, analizar, documentar y administrar las necesidades del sistema durante todo el proceso de desarrollo de software. Estas actividades garantizan que los requisitos permanezcan alineados con los objetivos del proyecto y puedan adaptarse a los cambios que surjan a lo largo del ciclo de vida del sistema [17].

Las principales etapas son:

- **Elicitación (Identificación):** consiste en recopilar las necesidades, expectativas y restricciones de los usuarios y demás interesados mediante técnicas como entrevistas, encuestas, observación, talleres o análisis documental [18].
- **Análisis y negociación:** los requisitos obtenidos se analizan para verificar su viabilidad técnica, consistencia, prioridad y posibles conflictos entre los diferentes interesados. En esta etapa también se negocian los cambios o ajustes necesarios [18].
- **Especificación:** los requisitos se documentan de forma estructurada en un documento de especificación (SRS), describiendo tanto los requisitos funcionales como los no funcionales, siguiendo estándares como el IEEE/ISO/IEC 29148 [18].
- **Validación:** se verifica que los requisitos sean completos, correctos, consistentes, verificables y que representen fielmente las necesidades de los usuarios y los objetivos del sistema [18].
- **Gestión de requisitos:** comprende el control de cambios, el mantenimiento de la trazabilidad, el seguimiento de versiones y la actualización de los requisitos durante todo el ciclo de vida del proyecto [18].

2.2.3. Requisitos funcionales

Los requisitos funcionales describen las funciones, servicios o comportamientos que el sistema debe proporcionar para satisfacer las necesidades de los usuarios y cumplir con los objetivos del proyecto. Estos requisitos especifican las acciones que el software debe realizar, las entradas que debe procesar, las respuestas que debe generar y las reglas de negocio que debe cumplir durante su funcionamiento [19].

Los requisitos funcionales representan las capacidades principales del sistema y sirven como base para el diseño, la implementación, las pruebas y la validación del software. Generalmente, se expresan mediante sentencias claras y verificables que indican el comportamiento esperado del sistema ante determinadas condiciones [16].

2.2.4. Requisitos no funcionales

Los requisitos no funcionales describen las características de calidad, restricciones y condiciones bajo las cuales el sistema debe operar. A diferencia de los requisitos funcionales, no especifican las funciones que el software debe realizar, sino los atributos que debe cumplir para garantizar un adecuado desempeño, seguridad, confiabilidad y facilidad de uso [20].

Estos requisitos establecen criterios relacionados con aspectos como el rendimiento, la seguridad, la disponibilidad, la usabilidad, la compatibilidad, la escalabilidad y la mantenibilidad del sistema. Su correcta definición es fundamental, ya que influyen directamente en la calidad del software y en la satisfacción de los usuarios finales [21].

Al igual que los requisitos funcionales, los requisitos no funcionales deben redactarse de forma clara, objetiva y verificable, de manera que puedan ser evaluados mediante pruebas o métricas específicas [22].

2.2.5. Casos de uso

Los casos de uso son una técnica de modelado utilizada para describir la interacción entre los actores (usuarios o sistemas externos) y el sistema con el propósito de alcanzar un objetivo específico. Cada caso de uso representa una funcionalidad que el sistema ofrece a un actor y permite documentar, de manera clara y estructurada, el comportamiento esperado del software desde la perspectiva del usuario [23].

Los casos de uso facilitan la comprensión de los requisitos funcionales, ya que muestran cómo los usuarios interactúan con el sistema para ejecutar determinadas tareas. Además, constituyen una herramienta de comunicación entre los interesados y el equipo de desarrollo, sirviendo como base para el diseño, la implementación y la elaboración de casos de prueba [24].

2.2.6. Historias de usuario

Las historias de usuario son descripciones breves y sencillas de una necesidad o funcionalidad del sistema, redactadas desde la perspectiva del usuario final. Su principal objetivo es expresar el valor que una determinada funcionalidad aporta al usuario, facilitando la comunicación entre los interesados y el equipo de desarrollo durante la planificación e implementación del software [25].

A diferencia de los requisitos funcionales, las historias de usuario se centran en las necesidades del usuario y no en los aspectos técnicos del sistema. Son ampliamente utilizadas en metodologías ágiles, como Scrum, donde forman parte del Product Backlog y sirven como base para la planificación de iteraciones o sprints [26]. Generalmente, una historia de usuario se redacta siguiendo la siguiente estructura:

Como [tipo de usuario], *quiero* [funcionalidad o necesidad], *para* [beneficio u objetivo].

Esta estructura permite identificar claramente quién solicita la funcionalidad, qué desea realizar y cuál es el beneficio esperado, facilitando la comprensión y posterior validación del requisito [27].

2.3. Documento ERS/SRS según IEEE/ISO/IEC 29148

El Documento de Especificación de Requisitos de Software (ERS), conocido internacionalmente como Software Requirements Specification (SRS), es un documento formal que describe de manera detallada los requisitos funcionales y no funcionales que debe cumplir un sistema de software. Su principal propósito es establecer una descripción clara, completa y verificable del comportamiento esperado del sistema antes de iniciar las etapas de diseño e implementación[18].

De acuerdo con el estándar ISO/IEC/IEEE 29148, el SRS constituye uno de los documentos más importantes dentro del proceso de Ingeniería de Requerimientos, ya que sirve como medio de comunicación entre clientes, usuarios, analistas, desarrolladores y demás interesados del proyecto [16]. Además, proporciona una base para las actividades de diseño, desarrollo, pruebas, mantenimiento y gestión de cambios [15].

El documento SRS no solo describe las funcionalidades que deberá ofrecer el sistema, sino también las restricciones, reglas de negocio, atributos de calidad, interfaces externas y demás condiciones necesarias para garantizar que el producto final satisfaga las necesidades de los interesados [14].

2.3.1. Objetivos del documento ERS/SRS

El documento de Especificación de Requisitos de Software tiene como finalidad establecer una descripción formal del sistema que sirva de referencia durante todo su ciclo de vida. Entre sus principales objetivos se encuentran:

- Documentar de manera clara y estructurada los requisitos funcionales y no funcionales del sistema [12].
- Servir como medio de comunicación entre clientes, usuarios, analistas, desarrolladores y demás interesados.
- Reducir ambigüedades e inconsistencias durante el desarrollo del software.
- Proporcionar una base para el diseño, implementación, pruebas y mantenimiento del sistema.
- Facilitar la validación y verificación de los requisitos mediante criterios objetivos.
- Permitir la trazabilidad de los requisitos desde su identificación hasta su implementación.
- Servir como documento contractual o de referencia para la aceptación del sistema [14].

2.3.2. Beneficios del documento ERS/SRS

La elaboración de un documento SRS aporta múltiples beneficios al proceso de desarrollo de software, ya que permite organizar la información del proyecto y reducir los riesgos asociados a una especificación deficiente [15].

Entre los principales beneficios destacan:

- Mejora la comunicación entre todos los participantes del proyecto.
- Disminuye la probabilidad de desarrollar funcionalidades incorrectas.
- Reduce los costos ocasionados por cambios tardíos en los requisitos.
- Facilita la planificación del proyecto y la estimación de recursos.
- Proporciona una base sólida para el diseño arquitectónico del sistema.
- Sirve como referencia para el desarrollo de casos de prueba.
- Favorece la gestión de cambios mediante mecanismos de trazabilidad.
- Contribuye a mejorar la calidad del software desarrollado.

2.3.3. Estructura recomendada según IEEE/ISO/IEC 29148

El estándar ISO/IEC/IEEE 29148 propone una estructura organizada para documentar los requisitos del sistema, permitiendo que la información sea comprensible, consistente y fácilmente mantenible [14].

Aunque la estructura puede adaptarse a las necesidades de cada proyecto, generalmente un documento SRS contiene los siguientes apartados:

Tabla 1. Estructura SRS.

Sección	Descripción
Introducción	Presenta el propósito, alcance, definiciones, referencias y organización del documento.
Descripción general	Describe el contexto del sistema, usuarios, restricciones, supuestos y características principales.
Requisitos específicos	Incluye los requisitos funcionales, no funcionales, interfaces, reglas de negocio y restricciones.
Modelos del sistema	Diagramas UML, modelos de datos, modelos funcionales y de comportamiento que complementan la especificación.
Trazabilidad	Relaciona los requisitos con los diferentes artefactos del proyecto para facilitar el seguimiento de cambios.
Anexos	Información complementaria, glosario, diagramas adicionales y documentos de apoyo.

2.3.4. Aplicación del documento ERS/SRS en UteqMarket

En el proyecto UteqMarket, el documento de Especificación de Requisitos de Software constituye la base para el desarrollo de la aplicación web destinada a la compra y venta de productos entre estudiantes, docentes y personal administrativo de la Universidad Técnica Estatal de Quevedo [13].

Dentro del documento se describen aspectos como:

- El objetivo del sistema.
- Los tipos de usuarios.
- Los requisitos funcionales relacionados con el registro de usuarios, autenticación mediante correo institucional, publicación de productos, búsqueda, filtrado, contacto entre usuarios y gestión de publicaciones.
- Los requisitos no funcionales relacionados con seguridad, rendimiento, disponibilidad y usabilidad.
- Las reglas de negocio, entre ellas la restricción de acceso únicamente mediante cuentas institucionales @uteq.edu.ec.
- Los modelos UML utilizados para representar la estructura de datos, los procesos y el comportamiento del sistema[6].

La utilización de un documento ERS/SRS en UteqMarket permitirá mantener una especificación organizada y consistente, facilitando el desarrollo del software, la validación de los requisitos y el control de futuras modificaciones.

Tabla 2. Contenido del SRS.

Capítulo del SRS	Contenido principal	Aplicación en UteqMarket
Introducción	Propósito, alcance, referencias y definiciones	Presenta el objetivo del sistema y el contexto universitario
Descripción general	Usuarios, restricciones y funciones generales	Estudiantes, docentes y personal administrativo; uso exclusivo de correo institucional
Requisitos funcionales	Funcionalidades del sistema	Registro, autenticación, publicación, búsqueda, filtros, mensajería y calificaciones
Requisitos no funcionales	Calidad del sistema	Seguridad, rendimiento, disponibilidad y usabilidad
Modelos del sistema	Diagramas UML y modelos de análisis	Casos de uso, ER, clases, DFD, actividades, estados y secuencia
Trazabilidad	Relación entre requisitos y modelos	Matriz RF–Modelos utilizada en la práctica
Anexos	Información complementaria	Tabla de atributos, diagramas y registro de defectos

2.4. Modelos de datos (Perspectiva de datos)

2.4.1. Modelado conceptual

El modelado conceptual es el proceso mediante el cual se representa de forma abstracta la estructura de un sistema antes de su implementación. Su principal objetivo es identificar los elementos fundamentales que conforman el dominio del problema y establecer las relaciones existentes entre ellos, sin considerar aspectos específicos de una base de datos o de un lenguaje de programación [6].

Durante esta etapa se construye una representación de alto nivel que permite comprender cómo se organiza la información del sistema y cómo interactúan los diferentes elementos que lo componen. El modelado conceptual facilita la comunicación entre analistas, desarrolladores y usuarios, ya que proporciona una visión clara y comprensible de la información que será administrada por el software [4].

Dentro del desarrollo de sistemas de información, los modelos conceptuales constituyen la base para el diseño lógico y físico de las bases de datos, además de servir como apoyo

para la construcción de modelos UML y otros artefactos utilizados durante el análisis y diseño del software [3].

2.4.2. Modelo entidad relación (ER)

El Modelo Entidad–Relación (ER) es una técnica de modelado conceptual propuesta por Peter Chen que permite representar gráficamente la estructura lógica de la información que administrará un sistema. Su objetivo principal es identificar las entidades del dominio, sus atributos y las relaciones existentes entre ellas, facilitando posteriormente el diseño de bases de datos relacionales [4].

El modelo ER constituye una de las herramientas más utilizadas durante el análisis de sistemas debido a su simplicidad y capacidad para representar de manera clara la organización de la información. Generalmente se representa mediante diagramas conocidos como Diagramas Entidad–Relación (DER), donde cada entidad, atributo y relación se representan utilizando símbolos estandarizados[4].

En proyectos de software, el modelo ER suele elaborarse antes del diseño físico de la base de datos, permitiendo detectar redundancias, inconsistencias y posibles problemas de organización de la información [4].

2.4.3. Entidades

Una entidad representa cualquier objeto, persona, lugar, evento o concepto del mundo real sobre el cual el sistema necesita almacenar información. Cada entidad posee un conjunto de atributos que describen sus características y permiten diferenciar una instancia de otra.

Las entidades pueden clasificarse como entidades fuertes, cuando poseen una clave primaria propia, o entidades débiles, cuando dependen de otra entidad para ser identificadas [5].

2.4.4. Relaciones

Las relaciones representan las asociaciones existentes entre dos o más entidades dentro del modelo de datos. Estas asociaciones describen la forma en que los elementos del sistema interactúan entre sí y permiten establecer reglas para mantener la integridad de la información almacenada [4].

Dependiendo de la naturaleza de la asociación, las relaciones pueden ser de uno a uno (1:1), uno a muchos (1:N) o muchos a muchos (N:M).

2.4.5. Cardinalidades

La cardinalidad indica el número mínimo y máximo de ocurrencias que una entidad puede tener respecto a otra dentro de una relación. Constituye uno de los elementos más importantes del modelo ER, ya que define las restricciones que deben respetarse al momento de almacenar la información [28].

Las cardinalidades más utilizadas son:

- Uno a uno (1:1)
- Uno a muchos (1:N)
- Muchos a muchos (N:M)

2.4.6. Notación Crow's Foot

La notación Crow's Foot es uno de los estándares gráficos más utilizados para representar diagramas Entidad-Relación. Su nombre proviene de la forma de "pata de cuervo" utilizada para representar el lado "muchos" de una relación [29].

Esta notación facilita la interpretación de las cardinalidades mediante símbolos gráficos que indican la obligatoriedad y multiplicidad de las relaciones entre entidades.

Actualmente es la notación utilizada por herramientas CASE como Visual Paradigm, PowerDesigner, MySQL Workbench y Oracle SQL Developer Data Modeler [4].

2.4.7. Normalización

La normalización es un proceso utilizado durante el diseño de bases de datos relacionales cuyo propósito es organizar la información de manera eficiente para reducir la redundancia de datos, evitar inconsistencias y garantizar la integridad de la información almacenada. Este proceso consiste en dividir la información en diferentes tablas relacionadas entre sí, siguiendo un conjunto de reglas conocidas como formas normales.

La aplicación de la normalización permite minimizar problemas como la duplicación de datos, las anomalías durante las operaciones de inserción, actualización y eliminación, así como facilitar el mantenimiento y la escalabilidad de la base de datos. En consecuencia, una base de datos normalizada mejora el rendimiento de las consultas, reduce el almacenamiento innecesario y asegura que la información permanezca consistente a lo largo del tiempo[29].

La normalización se lleva a cabo de manera progresiva, aplicando una serie de formas normales. Aunque existen formas normales superiores, como la Cuarta Forma Normal (4FN) y la Quinta Forma Normal (5FN), en la mayoría de aplicaciones empresariales resulta suficiente alcanzar la Tercera Forma Normal (3FN) [30].

Primera Forma Normal (1FN)

La Primera Forma Normal (1FN) establece que todos los atributos de una tabla deben contener valores atómicos, es decir, cada campo debe almacenar un único valor y no listas o conjuntos de datos. Además, cada registro debe ser único y no deben existir grupos repetitivos de información [30].

Segunda Forma Normal (2FN)

La Segunda Forma Normal (2FN) se alcanza cuando la tabla ya cumple con la Primera Forma Normal y, además, todos los atributos que no forman parte de la clave primaria dependen completamente de ella [30].

Esta forma normal busca eliminar las dependencias parciales, las cuales aparecen principalmente cuando la clave primaria está compuesta por varios atributos y alguno de los campos depende únicamente de una parte de dicha clave.

Tercera Forma Normal (3FN)

La Tercera Forma Normal (3FN) se alcanza cuando la tabla cumple con la Segunda Forma Normal y, además, ninguno de los atributos que no pertenecen a la clave primaria depende de otro atributo que tampoco sea clave.

Este tipo de dependencia se conoce como dependencia transitiva, ya que un atributo depende indirectamente de la clave primaria a través de otro atributo [30].

2.4.8. Llaves

Las llaves (Keys) son atributos o conjuntos de atributos utilizados para identificar de manera única cada registro dentro de una tabla y establecer relaciones entre las diferentes entidades de una base de datos relacional. Constituyen uno de los elementos fundamentales del diseño de bases de datos, ya que garantizan la integridad de la información, evitan registros duplicados y permiten relacionar correctamente los datos almacenados[31].

Durante el proceso de modelado de datos, las llaves permiten definir cómo interactúan las entidades entre sí y aseguran que cada registro pueda ser identificado de forma única. Asimismo, facilitan la implementación de restricciones de integridad referencial, garantizando que las relaciones entre tablas permanezcan consistentes durante las operaciones de inserción, actualización y eliminación de datos [29].

Existen diversos tipos de llaves; sin embargo, las más utilizadas durante el diseño de bases de datos son la llave primaria, la llave foránea y la llave candidata [31].

Llave primaria (Primary Key)

La llave primaria (Primary Key o PK) es el atributo, o conjunto de atributos, que identifica de manera única cada registro de una tabla. Su principal función es garantizar que no existan registros duplicados y permitir que cada elemento pueda ser localizado de forma inequívoca dentro de la base de datos [28].

Una llave primaria debe cumplir las siguientes características:

Sus valores deben ser únicos.

- No puede contener valores nulos (NULL).
- Debe permanecer estable durante el ciclo de vida del registro.
- Debe identificar únicamente a un registro dentro de la tabla.

En la mayoría de los sistemas se utiliza un identificador numérico autoincremental como llave primaria debido a su simplicidad y eficiencia.

Llave foránea (Foreign Key)

La llave foránea (Foreign Key o FK) es un atributo que hace referencia a la llave primaria de otra tabla. Su función principal es establecer relaciones entre entidades y mantener la integridad referencial de la base de datos.

Gracias a las llaves foráneas es posible representar asociaciones como uno a muchos (1:N) o muchos a muchos (N:M), evitando la duplicación innecesaria de información y garantizando que los datos relacionados existan previamente.

Por ejemplo, un producto siempre debe pertenecer a una categoría válida. Para ello, la entidad Producto almacena el atributo idCategoría, el cual hace referencia a la llave primaria de la entidad Categoría [28].

Llave candidata (Candidate Key)

Una llave candidata es cualquier atributo o conjunto de atributos que posee la capacidad de identificar de manera única cada registro dentro de una tabla y que, por tanto, podría ser seleccionado como llave primaria.

Es posible que una entidad tenga varias llaves candidatas; sin embargo, únicamente una de ellas será elegida como llave primaria. Las demás continúan siendo llaves candidatas y, en algunos casos, pueden implementarse como restricciones UNIQUE para evitar registros duplicados [29].

2.4.9. Diagrama de clases

El Diagrama de Clases UML es uno de los diagramas estructurales más importantes del Lenguaje Unificado de Modelado (UML). Su propósito es representar la estructura estática del sistema mediante clases, atributos, operaciones y relaciones entre objetos [6].

A diferencia del modelo ER, el diagrama de clases no solo representa la información que almacena el sistema, sino también el comportamiento de cada clase mediante métodos u operaciones. Este diagrama constituye la base para el diseño orientado a objetos y suele utilizarse como referencia para la implementación del software [6].

2.5. Modelos funcionales

2.5.1. Perspectiva funcional

La perspectiva funcional representa el comportamiento del sistema desde el punto de vista de las funciones o procesos que este debe realizar para satisfacer los requisitos establecidos. A diferencia de la perspectiva de datos, que se centra en la información almacenada, la perspectiva funcional describe cómo se transforma la información, cuáles son los procesos involucrados y cómo interactúan entre sí para producir los resultados esperados [32].

Su principal objetivo es modelar el funcionamiento interno del sistema mediante la representación de procesos, entradas, salidas y flujos de información, facilitando la comprensión de las actividades que ejecutará el software. Este tipo de modelado resulta

especialmente útil durante las etapas de análisis de requisitos, ya que permite identificar claramente las funciones principales del sistema antes de iniciar su implementación [32].

En Ingeniería de Software, la perspectiva funcional suele representarse mediante herramientas como los Diagramas de Flujo de Datos (DFD) y los Diagramas de Actividades UML, los cuales describen el flujo de información y la secuencia de actividades necesarias para ejecutar un proceso determinado [32].

2.5.2. Flujo de datos

El flujo de datos representa el movimiento de la información entre los diferentes procesos, entidades externas y almacenes de datos que conforman un sistema. Describe cómo los datos ingresan al sistema, cómo son transformados mediante distintos procesos y cómo son almacenados o enviados nuevamente hacia los usuarios u otros sistemas externos[33].

El análisis del flujo de datos permite comprender el recorrido que sigue la información desde su origen hasta su destino final, identificando las entradas, salidas, transformaciones y dependencias existentes entre los distintos procesos del sistema [33].

Generalmente, el flujo de datos se representa mediante flechas dentro de un Diagrama de Flujo de Datos (DFD), indicando la dirección en que viaja la información .

2.5.3. Diagrama de Flujo de Datos (DFD)

El Diagrama de Flujo de Datos (DFD) es una técnica de modelado utilizada para representar gráficamente cómo circula la información dentro de un sistema. Su objetivo es mostrar los procesos que transforman los datos, las entidades externas que interactúan con el sistema, los almacenes donde se conserva la información y los flujos de datos que conectan todos estos elementos [34].

A diferencia de otros diagramas UML, un DFD no representa objetos ni decisiones de programación, sino el recorrido que sigue la información durante la ejecución de los procesos del sistema.

Los principales elementos que conforman un DFD son:

- Entidad externa: representa actores o sistemas externos que intercambian información con el sistema.
- Proceso: transforma datos de entrada en datos de salida.
- Flujo de datos: representa el movimiento de la información entre procesos, entidades y almacenes.
- Almacén de datos: lugar donde la información es almacenada de forma permanente.

Los DFD pueden representarse en distintos niveles de detalle, permitiendo analizar progresivamente el funcionamiento del sistema.

2.5.4. Diagrama de Flujo de Datos Nivel 0

El DFD Nivel 0, también conocido como Diagrama de Contexto, ofrece una visión general del sistema representándolo como un único proceso principal. En este nivel únicamente se muestran las entidades externas que interactúan con el sistema y los flujos de información que intercambian con él [35].

Su objetivo es definir claramente los límites del sistema y establecer qué información entra y sale del mismo.

2.5.5. Diagrama de Flujo de Datos Nivel 1

El DFD Nivel 1 descompone el proceso principal representado en el Nivel 0 en varios subprocesos, mostrando con mayor detalle cómo circula la información dentro del sistema. En este nivel ya es posible identificar procesos específicos, almacenes de datos y los flujos existentes entre ellos [35].

2.5.6. DFD Esencial

El DFD Esencial representa el funcionamiento lógico del sistema sin considerar detalles tecnológicos o de implementación. Su propósito es describir únicamente qué hace el sistema, dejando de lado aspectos relacionados con interfaces, bases de datos, lenguajes de programación o tecnologías específicas. Este tipo de diagrama permite concentrarse exclusivamente en los procesos del negocio, facilitando el análisis de requisitos antes del diseño técnico. En esta práctica se utilizará el DFD Esencial para representar el proceso principal de Publicar producto dentro de UTeqMarket [35].

2.5.7. Diagrama de Actividades UML

El Diagrama de Actividades es un diagrama de comportamiento del Lenguaje Unificado de Modelado (UML) utilizado para representar el flujo secuencial de actividades que componen un proceso o caso de uso. Permite mostrar el orden en que se ejecutan las acciones, las decisiones que pueden tomarse durante el proceso y las posibles ejecuciones paralelas [36].

Este diagrama es especialmente útil para modelar procesos de negocio, describir algoritmos y representar el comportamiento de funcionalidades específicas del sistema desde el punto de vista operativo [37].

A diferencia del DFD, el diagrama de actividades enfatiza el flujo de control entre actividades y no únicamente el movimiento de la información [38].

2.6. Modelos de comportamiento

El comportamiento del sistema describe la manera en que un sistema de software responde a diferentes estímulos o eventos durante su ejecución. A diferencia de los modelos estructurales, que representan la organización de los datos o la arquitectura del sistema, los modelos de comportamiento se centran en mostrar cómo evolucionan los objetos a lo largo del tiempo y cómo interactúan entre sí para ejecutar una determinada funcionalidad [39].

Estos modelos permiten comprender la dinámica del sistema, identificando los cambios de estado de los objetos, la secuencia de mensajes intercambiados entre los diferentes componentes y las acciones que se ejecutan como respuesta a determinados eventos [39].

El modelado del comportamiento resulta especialmente útil durante la etapa de análisis y diseño, ya que facilita la validación de los requisitos funcionales, permite detectar inconsistencias antes de la implementación y mejora la comunicación entre analistas, desarrolladores y usuarios.

Dentro del Lenguaje Unificado de Modelado (UML), los diagramas más utilizados para representar el comportamiento del sistema son el Diagrama de Máquina de Estados y el Diagrama de Secuencia, los cuales ofrecen diferentes perspectivas del funcionamiento interno del software [39].

2.6.1. Máquina de estados

La Máquina de Estados (State Machine Diagram) es un diagrama de comportamiento de UML utilizado para representar los diferentes estados por los que puede pasar un objeto durante su ciclo de vida, así como las transiciones que ocurren entre dichos estados como consecuencia de eventos o condiciones específicas [40].

Este diagrama permite comprender cómo cambia el comportamiento de un objeto a medida que interactúa con otros elementos del sistema o responde a determinados eventos. Su principal objetivo es modelar la evolución temporal de una entidad, mostrando de manera clara las condiciones que provocan cada cambio de estado [40].

La máquina de estados resulta especialmente útil para modelar objetos cuyo comportamiento depende de su situación actual, como pedidos, publicaciones, usuarios, reservas o documentos [40].

2.6.2. Estados y eventos

Un estado representa una condición o situación específica en la que se encuentra un objeto durante un determinado momento de su ciclo de vida. Mientras un objeto permanece en un estado, puede ejecutar determinadas acciones, responder a eventos o esperar una transición hacia otro estado [41].

Los estados permiten representar el comportamiento dinámico de una entidad, indicando cómo evoluciona a medida que ocurren diferentes eventos dentro del sistema [41].

Los eventos son sucesos o estímulos que provocan un cambio de estado dentro de una máquina de estados. Un evento puede ser generado por una acción realizada por el usuario, por otro objeto del sistema o por la ocurrencia de una condición específica. Cuando se produce un evento, el sistema evalúa si existe una transición asociada al estado actual. En caso afirmativo, el objeto cambia al nuevo estado correspondiente y ejecuta las acciones definidas para dicha transición [41].

2.6.3. Diagrama de secuencia

El Diagrama de Secuencia es un diagrama de interacción de UML utilizado para representar el intercambio de mensajes entre los diferentes objetos o componentes del sistema durante la ejecución de una funcionalidad específica[42].

Su objetivo es mostrar el orden cronológico en que ocurren las interacciones entre los participantes, indicando qué objeto inicia una comunicación, cuál responde y qué operaciones se ejecutan durante el proceso. Este tipo de diagrama resulta especialmente útil para modelar casos de uso, validar requisitos funcionales y comprender la colaboración entre las distintas capas del sistema [42].

2.7. Interrelaciones entre las tres perspectivas

Durante el análisis y diseño de un sistema de software no es suficiente utilizar un único modelo para representar sus características, ya que cada modelo permite observar el sistema desde un punto de vista diferente. Por esta razón, la Ingeniería de Software emplea diversas perspectivas de modelado que, aunque poseen objetivos distintos, se complementan entre sí para ofrecer una representación integral del sistema [43].

En la presente práctica se consideran tres perspectivas fundamentales:

- Perspectiva de datos, orientada a representar la estructura de la información que administrará el sistema.
- Perspectiva funcional, enfocada en describir los procesos y el flujo de información entre las diferentes actividades.
- Perspectiva de comportamiento, encargada de representar la evolución de los objetos y las interacciones que ocurren durante la ejecución del sistema.

Estas perspectivas no deben interpretarse como modelos independientes, sino como representaciones complementarias de un mismo conjunto de requisitos. Cada requisito funcional puede analizarse desde las tres perspectivas, permitiendo comprender qué información utiliza, qué procesos ejecuta y cómo se comportan los objetos involucrados durante su ejecución [42].

2.7.1. Integración de los modelos

Los modelos desarrollados durante la Ingeniería de Requerimientos mantienen una estrecha relación entre sí. La información representada en un modelo sirve como base para la construcción de los demás, permitiendo conservar la consistencia durante todo el proceso de desarrollo [44].

Por ejemplo, una entidad identificada en el Modelo Entidad–Relación suele convertirse posteriormente en una clase dentro del Diagrama de Clases. De igual manera, un proceso representado en un Diagrama de Flujo de Datos puede detallarse mediante un Diagrama de Actividades, mientras que las interacciones entre los diferentes componentes involucrados en dicho proceso pueden representarse mediante un Diagrama de Secuencia.

Finalmente, los cambios que experimenta una entidad a lo largo de su ciclo de vida pueden describirse mediante una Máquina de Estados [44].

Esta relación entre modelos facilita la trazabilidad de los requisitos, ya que permite verificar que cada funcionalidad del sistema se encuentra representada desde las diferentes perspectivas de análisis [44].

3. Desarrollo de la practica

La revisión y refinamiento del Documento de Especificación de Requisitos de Software de UTeqMarket tiene como finalidad identificar defectos en la redacción de los requisitos y mejorar su calidad antes de utilizarlos como base para el modelado y desarrollo del sistema. Este proceso permite reducir ambigüedades, inconsistencias, omisiones y requisitos difíciles de comprobar.

De acuerdo con la rúbrica de la práctica, para alcanzar el nivel excelente se deben identificar y clasificar al menos cinco defectos, corregir los requisitos mediante la sintaxis EARS y completar la tabla de atributos para todos los requisitos funcionales y no funcionales.

3.1. Revisión del SRS

La revisión del SRS de UTeqMarket se realizó mediante una inspección manual y sistemática de los requisitos funcionales y no funcionales previamente definidos. Cada requisito fue analizado individualmente para determinar si expresaba con claridad el comportamiento esperado del sistema y si podía ser comprobado mediante una prueba objetiva.

Durante el proceso se consideraron criterios de calidad relacionados con la claridad, completitud, consistencia, necesidad, factibilidad, verificabilidad y trazabilidad. También se revisó que cada requisito tuviera un único significado y que no mezclara varias funciones dentro de una misma redacción.

El procedimiento de revisión se desarrolló de la siguiente manera:

- Se recopiló la lista completa de requisitos funcionales y no funcionales del proyecto.
- Se comprobó que cada requisito tuviera un identificador único.
- Se revisó que las descripciones utilizaran un lenguaje claro y preciso.
- Se identificaron palabras ambiguas, como «rápido», «seguro», «fácil», «adecuado» o «correctamente».
- Se verificó que los requisitos no combinaran varias funcionalidades independientes.
- Se revisó la existencia de condiciones, eventos, respuestas del sistema y criterios de aceptación.
- Se comprobó que los requisitos no se contradijeran entre sí.

- Se verificó que los requisitos no funcionales incluyeran valores medibles.
- Se aplicaron patrones EARS para mejorar la redacción.
- Se relacionaron los requisitos corregidos con sus atributos de prioridad, fuente, estabilidad, estado y verificabilidad.

La revisión permitió detectar que varios requisitos iniciales describían la funcionalidad de forma demasiado general. Por ejemplo, el requisito «El sistema permitirá publicar productos» no indicaba quién podía realizar la acción, qué información debía registrarse, qué validaciones se ejecutarían ni cuál sería la respuesta final del sistema.

También se identificaron requisitos no funcionales con expresiones subjetivas. Un requisito como «El sistema deberá ser rápido» no puede comprobarse objetivamente porque no define un tiempo máximo de respuesta ni las condiciones bajo las cuales debe evaluarse.

Como resultado, los requisitos fueron reformulados para incluir actores, eventos, condiciones, restricciones y respuestas verificables. Asimismo, se estableció una redacción uniforme basada en expresiones como «el sistema deberá», evitando términos opcionales como «podría», «debería» o «se espera que».

3.2. Defectos encontrados

Para clasificar los problemas detectados se utilizaron los siguientes tipos de defectos:

- **Ambigüedad:** el requisito puede interpretarse de más de una manera.
- **Incompletitud:** faltan datos, condiciones o resultados necesarios.
- **No verificable:** no puede comprobarse mediante una prueba objetiva.
- **Inconsistencia:** contradice otro requisito o regla del sistema.
- **Requisito compuesto:** contiene varias funciones independientes.
- **Falta de restricción:** no establece límites o reglas necesarias.
- **Falta de trazabilidad:** no se relaciona claramente con una necesidad o fuente.
- **Orientación a la solución:** especifica innecesariamente cómo debe implementarse la función.

Tabla 3. Defectos encontrados.

ID	Requisito o defecto identificado	Tipo de defecto	Corrección propuesta
D-01	«El sistema permitirá publicar productos». No identifica al actor, los datos requeridos ni las validaciones.	Incompletitud	Especificar que el usuario debe estar autenticado, indicar los campos obligatorios y definir que el sistema validará los datos antes de

			guardar la publicación.
D-02	«El sistema validará el correo institucional». No indica el dominio permitido ni el proceso de validación.	Ambigüedad	Indicar que el correo debe pertenecer al dominio @uteq.edu.ec y que la cuenta debe verificarse mediante un enlace o mecanismo de autenticación.
D-03	«La aplicación debe ser rápida». No contiene un valor medible.	No verificable	Establecer un tiempo máximo de carga, por ejemplo, tres segundos bajo condiciones normales de uso.
D-04	«El usuario podrá contactar al vendedor». No especifica el medio de contacto ni las condiciones para mostrar los datos.	Incompletitud	Definir que el comprador autenticado podrá abrir WhatsApp mediante el número registrado por el vendedor desde la página de detalle del producto.
D-05	«El sistema permitirá registrar cualquier número de WhatsApp». Esto puede aceptar letras, números incompletos o formatos inválidos.	Falta de restricción	Validar que el número contenga únicamente dígitos, corresponda a un formato permitido y tenga una longitud válida.
D-06	«El usuario podrá ingresar el precio del producto». No establece límites ni formato.	Falta de restricción	Definir que el precio debe ser numérico, positivo, tener un máximo establecido y permitir hasta dos decimales.
D-07	«El sistema permitirá buscar y filtrar productos por nombre, categoría, precio y ubicación». Contiene varias	Requisito compuesto	Separar la búsqueda por texto y los filtros por categoría, precio u otros criterios en requisitos independientes.

	funciones en una sola sentencia.		
D-08	«El sistema deberá ser seguro y fácil de usar». Incluye dos atributos de calidad subjetivos y diferentes.	Ambigüedad y requisito compuesto	Dividirlo en requisitos de seguridad y usabilidad, incorporando medidas verificables para cada uno.
D-09	«Los usuarios podrán registrarse con cualquier correo, pero solo se aceptarán correos UTEQ». Las dos condiciones se contradicen.	Inconsistencia	Establecer que únicamente se permitirá completar el registro con una cuenta institucional válida.
D-10	«El sistema utilizará Firebase para validar usuarios». Se enfoca en una tecnología concreta y no en la necesidad.	Orientación a la solución	Redactar el requisito indicando el resultado esperado: autenticar y verificar la cuenta institucional. Firebase puede registrarse como restricción tecnológica o decisión de diseño.

3.3. Tabla completa de atributos

3.3.1. Requisitos funcionales

Tabla 4. Requerimientos funcionales.

ID	Descripción	Tipo	MoS CoW	Fuente	Estabilidad	Estado	Verificable
RF-001	Registro de cuenta	RF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RF-002	Restricción de dominio institucional	RF	Must	Requisitos institucionales	Alta	Aprobado	Sí
RF-003	Unicidad del correo institucional	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RF-004	Rechazo de correos desechables	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí

RF-005	Validación de contraseña	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RF-006	Confirmación de contraseña	RF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RF-007	Verificación del correo electrónico	RF	Must	Requisitos institucionales	Alta	Aprobado	Sí
RF-008	Vigencia del token de verificación	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RF-009	Activación de cuenta	RF	Must	Requisitos institucionales	Alta	Aprobado	Sí
RF-010	Inicio de sesión	RF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RF-011	Cierre de sesión	RF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RF-012	Control de acceso por roles	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RF-013	Consulta pública del catálogo	RF	Should	Usuarios UTEQ	Alta	Aprobado	Sí
RF-014	Visualización de información resumida	RF	Should	Usuarios UTEQ	Alta	Aprobado	Sí
RF-015	Búsqueda textual de publicaciones	RF	Should	Usuarios UTEQ	Alta	Aprobado	Sí
RF-016	Filtrado de publicaciones	RF	Should	Usuarios UTEQ	Alta	Aprobado	Sí
RF-017	Ordenamiento del catálogo	RF	Should	Usuarios UTEQ	Media	Aprobado	Sí
RF-018	Persistencia de criterios de búsqueda	RF	Should	Equipo de desarrollo	Media	Aprobado	Sí
RF-019	Visualización del detalle de publicación	RF	Should	Usuarios UTEQ	Alta	Aprobado	Sí
RF-020	Consulta del perfil público del vendedor	RF	Should	Usuarios UTEQ	Media	Aprobado	Sí
RF-021	Creación de publicaciones	RF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RF-022	Asociación automática del propietario	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí

RF-023	Validación del contenido de la publicación	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RF-024	Validación del precio	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RF-025	Validación del número de WhatsApp	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RF-026	Reutilización de datos de contacto	RF	Should	Usuarios UTEQ	Media	Aprobado	Sí
RF-027	Asignación del estado inicial	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RF-028	Consulta de publicaciones propias	RF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RF-029	Edición de publicaciones	RF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RF-030	Cambio de estado de publicaciones	RF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RF-031	Eliminación de publicaciones	RF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RF-032	Marcado de producto como vendido	RF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RF-033	Marcado de producto como reservado	RF	Should	Usuarios UTEQ	Media	Aprobado	Sí
RF-034	Gestión de categorías	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RF-035	Asociación de publicaciones a categorías	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RF-036	Gestión de favoritos	RF	Could	Usuarios UTEQ	Media	Aprobado	Sí
RF-037	Agregar publicaciones a favoritos	RF	Could	Usuarios UTEQ	Media	Aprobado	Sí
RF-038	Eliminar publicaciones de favoritos	RF	Could	Usuarios UTEQ	Media	Aprobado	Sí
RF-039	Visualización de	RF	Could	Usuarios UTEQ	Media	Aprobado	Sí

	publicaciones favoritas						
RF-040	Contacto con el vendedor mediante WhatsApp	RF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RF-041	Generación automática del enlace de WhatsApp	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RF-042	Visualización del número de contacto	RF	Should	Usuarios UTEQ	Media	Aprobado	Sí
RF-043	Calificación de vendedores	RF	Could	Usuarios UTEQ	Media	Aprobado	Sí
RF-044	Registro de comentarios	RF	Could	Usuarios UTEQ	Media	Aprobado	Sí
RF-045	Consulta del historial de calificaciones	RF	Could	Usuarios UTEQ	Media	Aprobado	Sí
RF-046	Cálculo de reputación del usuario	RF	Should	Equipo de desarrollo	Alta	Aprobado	Sí
RF-047	Gestión del perfil de usuario	RF	Should	Usuarios UTEQ	Alta	Aprobado	Sí
RF-048	Actualización de información personal	RF	Should	Usuarios UTEQ	Media	Aprobado	Sí
RF-049	Actualización de fotografía de perfil	RF	Could	Usuarios UTEQ	Baja	Aprobado	Sí
RF-050	Administración de usuarios	RF	Must	Administrador	Alta	Aprobado	Sí
RF-051	Bloqueo y desbloqueo de usuarios	RF	Must	Administrador	Alta	Aprobado	Sí
RF-052	Gestión de reportes de publicaciones	RF	Should	Administrador	Media	Aprobado	Sí
RF-053	Eliminación de publicaciones reportadas	RF	Must	Administrador	Alta	Aprobado	Sí
RF-054	Consulta de estadísticas generales	RF	Could	Administrador	Baja	Aprobado	Sí

RF-055	Registro de actividad del sistema	RF	Should	Equipo de desarrollo	Alta	Aprobado	Sí
RF-056	Notificaciones al usuario	RF	Should	Usuarios UTEQ	Media	Aprobado	Sí
RF-057	Gestión de sesiones activas	RF	Should	Equipo de desarrollo	Alta	Aprobado	Sí
RF-058	Recuperación de contraseña	RF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RF-059	Restablecimiento seguro de contraseña	RF	Must	Equipo de desarrollo	Alta	Aprobado	Sí

3.3.2. Requisitos no funcionales

Tabla 5. Requerimientos no funcionales.

ID	Descripción	Tipo	MoSCoW	Fuente	Estabilidad	Estado	Verificable
RNF-001	Comunicación segura mediante HTTPS	RNF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RNF-002	Almacenamiento seguro de contraseñas	RNF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RNF-003	Gestión segura de sesiones	RNF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RNF-004	Control de autorización del servidor	RNF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RNF-005	Protección contra ataques CSRF	RNF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RNF-006	Protección contra inyección SQL	RNF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RNF-007	Validación de entradas del usuario	RNF	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RNF-008	Disponibilidad del sistema	RNF	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RNF-009	Tiempo máximo de respuesta	RNF	Must	Usuarios UTEQ	Alta	Aprobado	Sí

RN F-010	Tiempo de carga del catálogo	RN F	Should	Usuarios UTEQ	Alta	Aprobado	Sí
RN F-011	Optimización de consultas	RN F	Should	Equipo de desarrollo	Media	Aprobado	Sí
RN F-012	Escalabilidad del sistema	RN F	Should	Equipo de desarrollo	Media	Aprobado	Sí
RN F-013	Compatibilidad con navegadores modernos	RN F	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RN F-014	Diseño responsive	RN F	Must	Usuarios UTEQ	Alta	Aprobado	Sí
RN F-015	Compatibilidad con dispositivos móviles	RN F	Should	Usuarios UTEQ	Alta	Aprobado	Sí
RN F-016	Accesibilidad de la interfaz	RN F	Should	Usuarios UTEQ	Media	Aprobado	Sí
RN F-017	Facilidad de uso de la interfaz	RN F	Should	Usuarios UTEQ	Media	Aprobado	Sí
RN F-018	Consistencia de la interfaz gráfica	RN F	Should	Equipo de desarrollo	Alta	Aprobado	Sí
RN F-019	Arquitectura modular	RN F	Should	Equipo de desarrollo	Alta	Aprobado	Sí
RN F-020	Mantenibilidad del código	RN F	Should	Equipo de desarrollo	Alta	Aprobado	Sí
RN F-021	Documentación técnica del sistema	RN F	Should	Equipo de desarrollo	Media	Aprobado	Sí
RN F-022	Registro de errores (logs)	RN F	Should	Equipo de desarrollo	Alta	Aprobado	Sí
RN F-023	Copias de seguridad de la información	RN F	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RN F-024	Recuperación ante fallos	RN F	Must	Equipo de desarrollo	Alta	Aprobado	Sí

RN F-025	Integración con Firebase Authentication	RN F	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RN F-026	Integración con Cloudinary	RN F	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RN F-027	Integración con WhatsApp	RN F	Should	Usuarios UTEQ	Media	Aprobado	Sí
RN F-028	Cumplimiento de estándares web	RN F	Should	Equipo de desarrollo	Media	Aprobado	Sí
RN F-029	Validación de formatos de archivos	RN F	Should	Equipo de desarrollo	Alta	Aprobado	Sí
RN F-030	Restricción del tamaño de imágenes	RN F	Should	Equipo de desarrollo	Media	Aprobado	Sí
RN F-031	Conservación de la integridad de los datos	RN F	Must	Equipo de desarrollo	Alta	Aprobado	Sí
RN F-032	Auditoría de acciones críticas	RN F	Could	Administrador	Media	Aprobado	Sí
RN F-033	Escalabilidad para futuras funcionalidades	RN F	Could	Equipo de desarrollo	Baja	Aprobado	Sí

3.4. Modelo de datos

3.4.1. Identificación de entidades y relaciones

Identificación de entidades

Relaciones M:N fueron resueltas mediante entidades asociativas.

USUARIO

Atributo	Tipo	Restricción
idUsuario	BIGINT	PK, NOT NULL
nombreUsuario	VARCHAR(120)	NOT NULL
correoUsuario	VARCHAR(180)	UNIQUE, NOT NULL

Atributo	Tipo	Restricción
contrasenaHashUsuario	VARCHAR(400)	NOT NULL
rolUsuario	VARCHAR(20)	NOT NULL
activoUsuario	BOOLEAN	NOT NULL
telefonoUsuario	VARCHAR(24)	NOT NULL
correoConfirmacion	BOOLEAN	NOT NULL
tokenVerificacionUsuario	VARCHAR(400)	NULL
expiracionTokenUsuario	DATETIME	NULL
fechaRegistroUsuario	DATETIME	NOT NULL

CATEGORIA

Atributo	Tipo	Restricción
idCategoria	INT	PK, NOT NULL
nombreCategoria	VARCHAR(80)	NOT NULL
slugCategoria	VARCHAR(40)	UNIQUE, NOT NULL
iconoCategoria	VARCHAR(80)	NOT NULL

PUBLICACION

Atributo	Tipo	Restricción
idPublicacion	BIGINT	PK, NOT NULL
tituloPublicacion	VARCHAR(140)	NOT NULL
descripcionPublicacion	VARCHAR(900)	NOT NULL
precioPublicacion	DECIMAL(10,2)	NOT NULL
ubicacionPublicacion	VARCHAR(120)	NOT NULL
telefonoPublicacion	VARCHAR(24)	NOT NULL
estadoPublicacion	VARCHAR(20)	NOT NULL
fechaRegistroPublicacion	DATETIME	NOT NULL

Atributo	Tipo	Restricción
idCategoriaPublicacion	INT	FK, NOT NULL
idUsuarioPublicacion	BIGINT	FK, NULL

IMAGENPUBLICACION

Atributo	Tipo	Restricción
idImagen	BIGINT	PK, NOT NULL
idPublicacionImagen	BIGINT	FK, NOT NULL
urlImagen	VARCHAR(600)	NOT NULL
ordenImagen	INT	NOT NULL
portadaImagen	BOOLEAN	NOT NULL

FAVORITO

Atributo	Tipo	Restricción
idFavorito	INT	PK, NOT NULL
idUsuarioFavorito	BIGINT	FK, NOT NULL, UNIQUE compuesto
idPublicacionFavorito	BIGINT	FK, NOT NULL, UNIQUE compuesto
fechaFavorito	DATETIME	NOT NULL

CONTACTO

Atributo	Tipo	Restricción
idContacto	BIGINT	PK, NOT NULL
idPublicacionContacto	BIGINT	FK, NOT NULL, UNIQUE compuesto
idUsuarioContacto	BIGINT	FK, NOT NULL, UNIQUE compuesto

Atributo	Tipo	Restricción
fechaContacto	DATETIME	NOT NULL

CALIFICACION

Atributo	Tipo	Restricción
idCalificacion	BIGINT	PK, NOT NULL
idPublicacionCalificacion	BIGINT	FK, NOT NULL, UNIQUE compuesto
idUsuarioCalificacion	BIGINT	FK, NOT NULL, UNIQUE compuesto
puntuacionCalificacion	INT	NOT NULL, CHECK 1..5
comentarioCalificacion	VARCHAR(500)	NOT NULL
fechaCalificacion	DATETIME	NOT NULL

REPORTE

Atributo	Tipo	Restricción
idReporte	BIGINT	PK, NOT NULL
idPublicacionReporte	BIGINT	FK, NOT NULL
idUsuarioReporte	BIGINT	FK, NOT NULL
motivoReporte	VARCHAR(30)	NOT NULL
descripcionReporte	VARCHAR(700)	NOT NULL
estadoReporte	VARCHAR(20)	NOT NULL
fechaReporte	DATETIME	NOT NULL

Identificación de relaciones

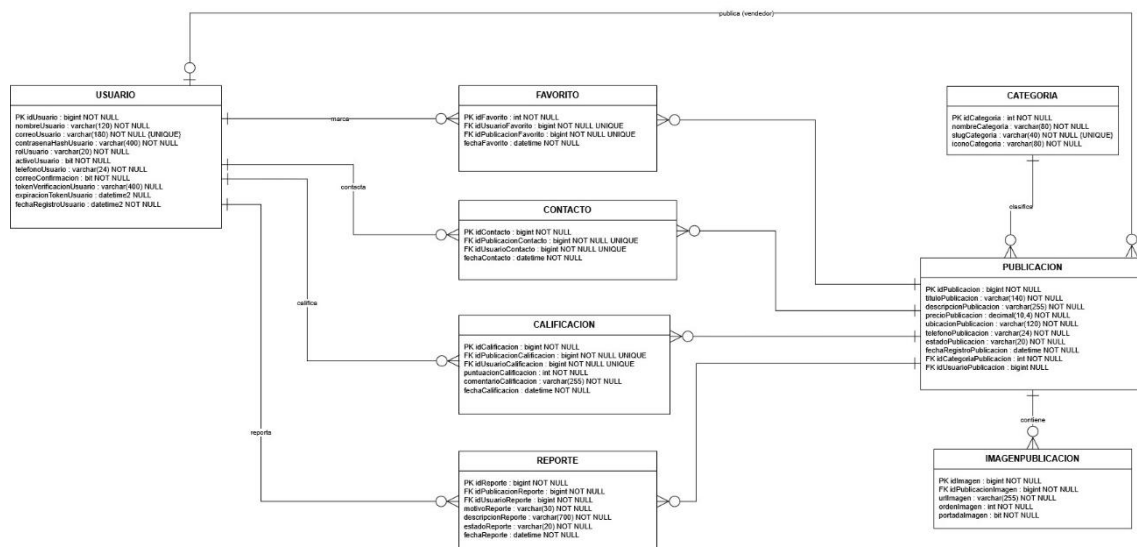
Nombre de la relación	Entidad A	Card. A	Entidad B	Card. B	Cardinalidad global	Participación (A/B)
clasifica	Categoria	1	Publicacion	o<	1:N	Total / Parcial

Nombre de la relación	Entidad A	Card. A	Entidad B	Card. B	Cardinalidad global	Participación (A/B)
publica (vendedor)	Usuario	o	Publicacion	o<	1:N	Parcial / Parcial
contiene	Publicacion		ImagenPublicacion	o<	1:N	Total / Parcial
marca	Usuario	o	Favorito	o<	1:N	Parcial / Parcial
es marcada	Publicacion	o	Favorito	o<	1:N	Parcial / Parcial
contacta	Usuario	o	Contacto	o<	1:N	Parcial / Parcial
recibe contacto	Publicacion	o	Contacto	o<	1:N	Parcial / Parcial
califica	Usuario	o	Calificacion	o<	1:N	Parcial / Parcial
recibe calificacion	Publicacion	o	Calificacion	o<	1:N	Parcial / Parcial
reporta	Usuario	o	Reporte	o<	1:N	Parcial / Parcial
es reportada	Publicacion	o	Reporte	o<	1:N	Parcial / Parcial

Relaciones M:N identificadas y resueltas mediante entidad asociativa (mínimo exigido por la rúbrica: 2): Usuario × Publicacion vía Favorito, Contacto, Calificacion y Reporte — 4 relaciones N:M en total, cada una con atributos propios en su entidad asociativa (fechaFavorito; fechaContacto; puntuacionCalificacion, comentarioCalificacion, fechaCalificacion; motivoReporte, descripcionReporte, estadoReporte, fechaReporte).

3.4.2. Diagrama ER

Ilustración 1. Diagrama entidad relación.



Fuente: draw.io

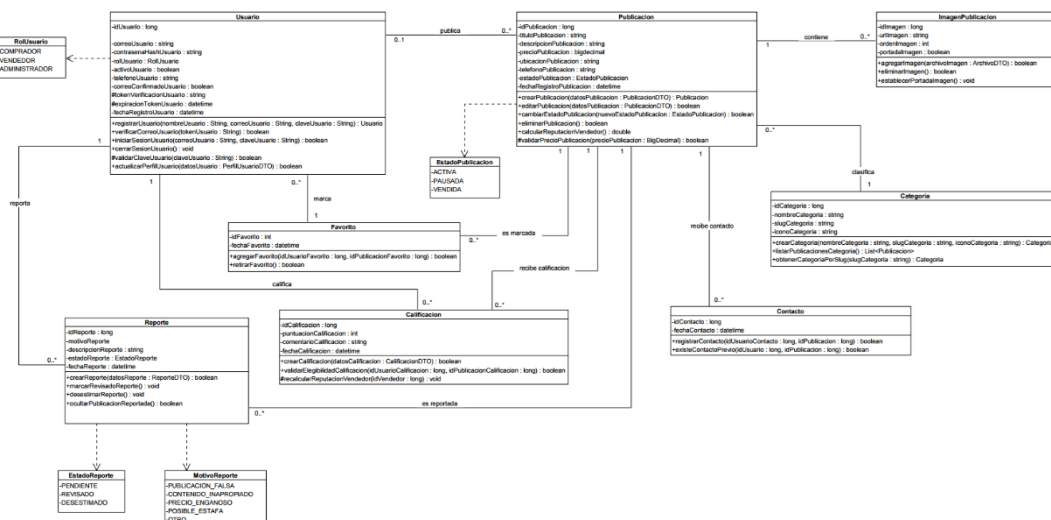
3.4.3. Verificación de 3FN

El modelo Entidad–Relación de UTeqMarket cumple con la Tercera Forma Normal (3FN) porque la información se encuentra organizada en entidades independientes, donde cada una representa un único concepto del dominio del problema, como Usuario, Publicación, Categoría, Favorito, Contacto, Calificación, Reporte e ImagenPublicación. Todas las entidades poseen una clave primaria que identifica de forma única cada registro, y sus atributos dependen exclusivamente de dicha clave, evitando dependencias parciales. Asimismo, los datos se almacenan de forma atómica, sin grupos repetitivos ni atributos multivaluados, satisfaciendo los principios de la Primera y Segunda Forma Normal.

Además, el modelo elimina las dependencias transitivas mediante el uso de entidades especializadas relacionadas por llaves foráneas. Por ejemplo, la información de las categorías se almacena únicamente en la entidad CATEGORIA, mientras que PUBLICACION conserva únicamente la llave foránea idCategoriaPublicacion, evitando repetir el nombre, icono o identificador de la categoría en cada publicación. De igual manera, las imágenes, favoritos, contactos, calificaciones y reportes se administran en tablas independientes, reduciendo la redundancia de datos, garantizando la integridad referencial y facilitando el mantenimiento, la actualización y la escalabilidad de la base de datos. Estos aspectos evidencian que el modelo cumple con los criterios establecidos para la Tercera Forma Normal (3FN).

3.4.4. Diagrama de clases

Ilustración 2. Diagrama de clases.



Fuente: draw.io

El diagrama de clases de UTEQMarket representa la estructura estática del sistema mediante las clases Usuario, Publicación, Categoría, ImagenPublicación, Favorito,

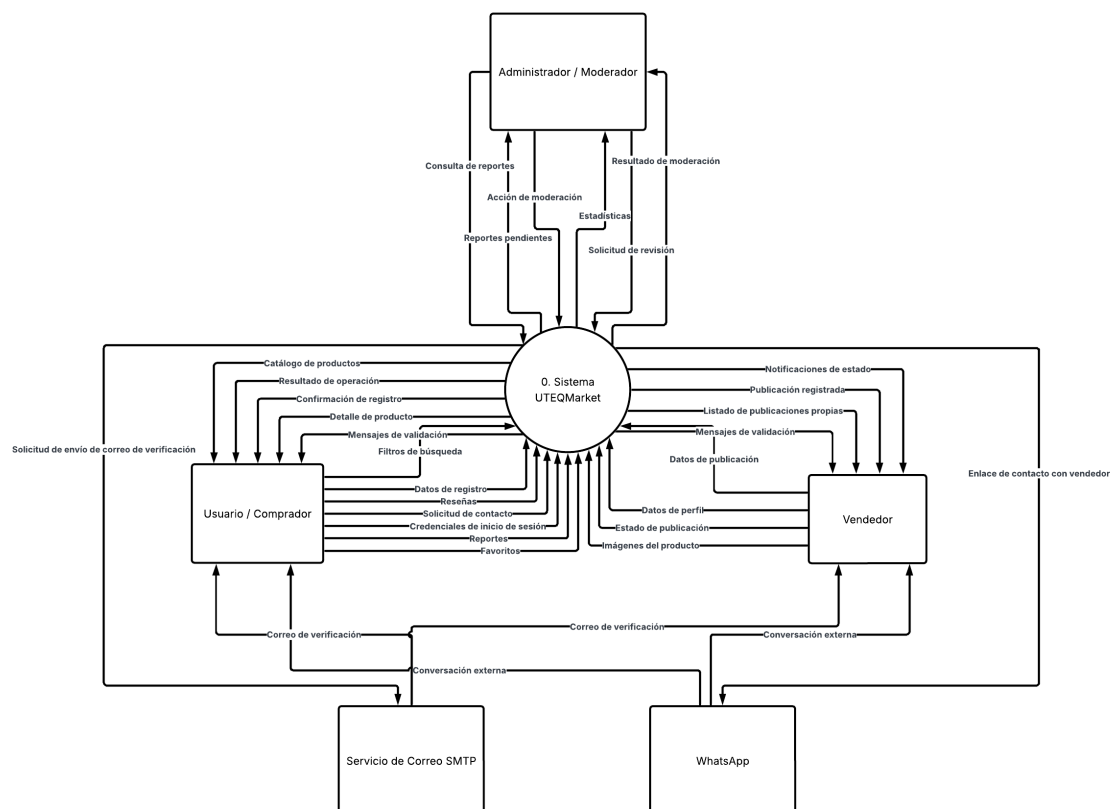
Contacto, Calificación y Reporte, junto con sus atributos, operaciones y relaciones. La clase Usuario concentra la información de la cuenta, autenticación, verificación y perfil, mientras que Publicación administra los datos principales del producto o servicio, su precio, ubicación, estado y operaciones de creación, edición y cambio de estado. Además, se utilizan enumeraciones como RolUsuario, EstadoPublicación, EstadoReporte y MotivoReporte, lo que permite restringir los valores posibles y mantener mayor consistencia en el modelo.

Las asociaciones reflejan las reglas principales del negocio. Un usuario puede publicar varias publicaciones, marcar varias como favoritas, registrar contactos, emitir calificaciones y generar reportes. Cada publicación pertenece a una categoría, puede contener varias imágenes y puede recibir múltiples contactos, calificaciones, favoritos y reportes. Las clases intermedias Favorito, Contacto, Calificación y Reporte funcionan como clases asociativas, ya que almacenan información propia de la relación entre un usuario y una publicación, como fechas, puntuaciones, comentarios, motivos y estados. En conjunto, el diagrama muestra una separación adecuada de responsabilidades, incorpora cardinalidades, visibilidades, tipos de datos y operaciones por clase, y sirve como base para implementar la lógica orientada a objetos del sistema de forma coherente con el modelo de datos y los requisitos funcionales de UTEQMarket.

3.5. Modelo funcional

3.5.1. DFD nivel 0

Ilustración 3. Diagrama de flujo de datos nivel 0.



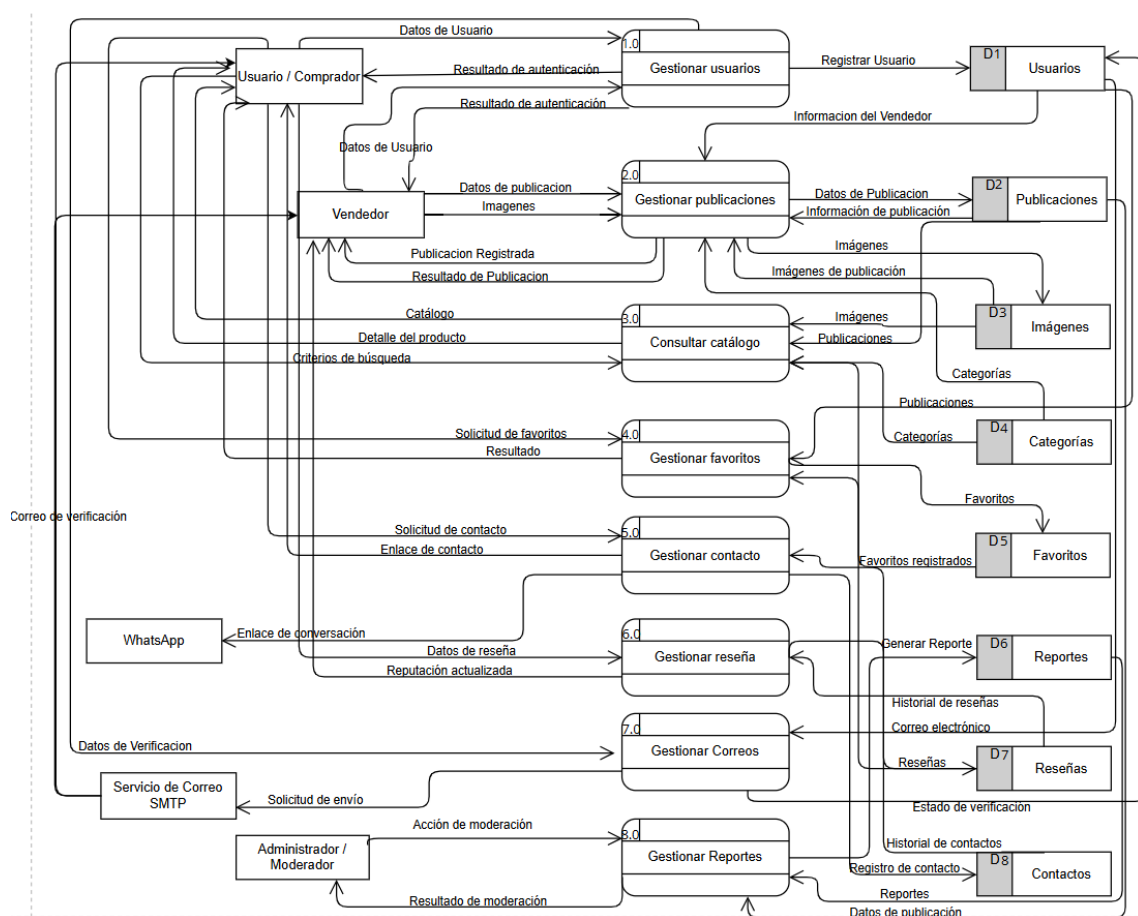
Fuente: visual-paradigm.com

El DFD de Nivel 0 de UTEQMarket presenta una visión general del sistema como un único proceso central denominado “0. Sistema UTEQMarket”, el cual interactúa con cinco entidades externas: Usuario/Comprador, Vendedor, Administrador/Moderador, Servicio de Correo SMTP y WhatsApp. Este nivel permite delimitar claramente el alcance del sistema y mostrar los principales flujos de información que entran y salen de la plataforma, sin detallar todavía los procesos internos ni los almacenes de datos. El usuario o comprador envía datos de registro, credenciales de inicio de sesión, filtros de búsqueda, reseñas, solicitudes de contacto, reportes y favoritos; a cambio recibe confirmaciones, mensajes de validación, resultados de operaciones, catálogo, detalles de productos y correos de verificación. Por su parte, el vendedor proporciona datos de perfil, información e imágenes de publicaciones y cambios de estado, mientras recibe confirmaciones, listados de publicaciones propias, notificaciones y resultados de validación.

El diagrama también representa las funciones de control y soporte externo. El Administrador/Moderador consulta reportes, estadísticas y solicitudes de revisión, y envía acciones de moderación, recibiendo posteriormente los resultados correspondientes. El Servicio de Correo SMTP participa en el envío de mensajes de verificación de cuenta, mientras que WhatsApp funciona como plataforma externa para iniciar la conversación entre compradores y vendedores. En conjunto, el diagrama refleja adecuadamente las principales entradas y salidas de UTEQMarket, evidencia la relación del sistema con sus actores y servicios externos, y sirve como base para elaborar el DFD Nivel 1, donde el proceso central deberá descomponerse en subprocesos como autenticación, gestión de publicaciones, búsqueda, contacto, reputación y moderación.

3.5.2. DFD nivel 1

Ilustración 4. Diagrama de flujo de datos nivel 1.



Fuente: visual-paradigm.com

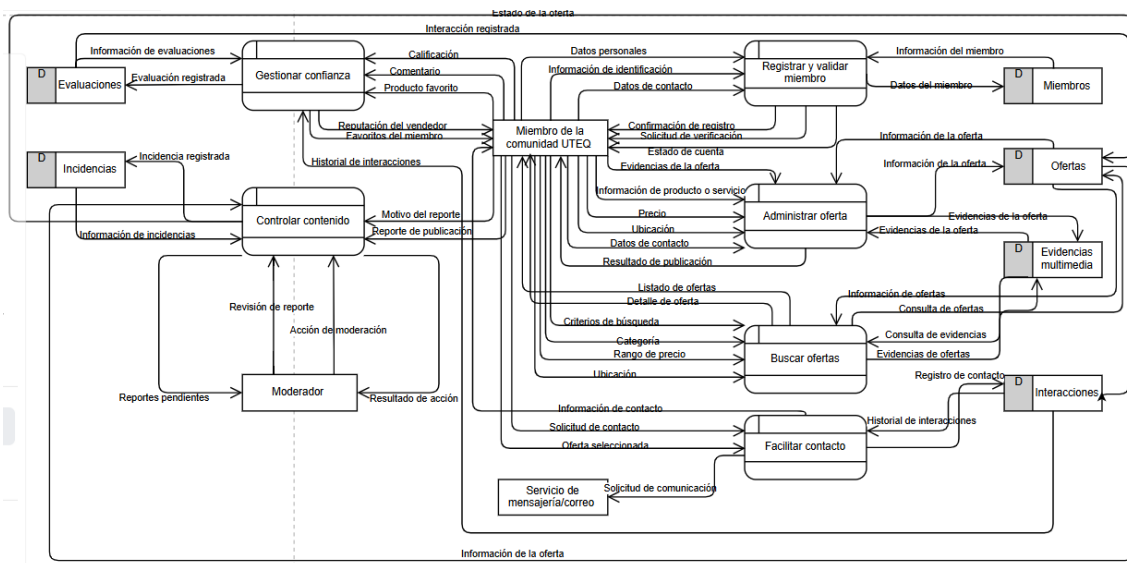
El DFD Nivel 1 de UTEQMarket descompone el proceso general mostrado en el DFD Nivel 0 en ocho procesos principales que representan las funcionalidades esenciales del sistema: Gestionar usuarios (1.0), Gestionar publicaciones (2.0), Consultar catálogo (3.0), Gestionar favoritos (4.0), Gestionar contacto (5.0), Gestionar reseñas (6.0), Gestionar correos (7.0) y Gestionar reportes (8.0). A diferencia del DFD Nivel 0, este diagrama permite visualizar con mayor detalle cómo circula la información entre los actores externos, los procesos internos y los almacenes de datos. Además de los actores Usuario/Comprador, Vendedor, Administrador/Moderador, Servicio de Correo SMTP y WhatsApp, se incorporan ocho almacenes de datos (Usuarios, Publicaciones, Imágenes, Categorías, Favoritos, Reportes, Reseñas y Contactos), donde se registra de forma permanente la información generada por cada proceso.

Cada proceso cumple una función específica dentro de la plataforma. Gestionar usuarios administra el registro, autenticación y almacenamiento de cuentas; Gestionar publicaciones permite crear, editar y actualizar las publicaciones junto con sus imágenes; Consultar catálogo recupera la información de publicaciones, categorías e imágenes para mostrar los resultados de búsqueda; Gestionar favoritos administra las publicaciones marcadas por los usuarios; Gestionar contacto registra las solicitudes de

comunicación y genera el enlace hacia WhatsApp; Gestionar reseñas almacena las calificaciones y actualiza la reputación de los vendedores; Gestionar correos coordina el envío de mensajes de verificación mediante el servicio SMTP; y Gestionar reportes permite que el administrador consulte y modere las publicaciones reportadas. En conjunto, el DFD Nivel 1 proporciona una representación detallada del funcionamiento interno de UTEQMarket, evidenciando cómo cada proceso intercambia información con los actores y almacenes de datos para cumplir los requisitos funcionales definidos en la especificación del sistema.

3.5.3. DFD esencial

Ilustración 5. Diagrama de flujo de datos esencial.



Fuente: visual-paradigm.com

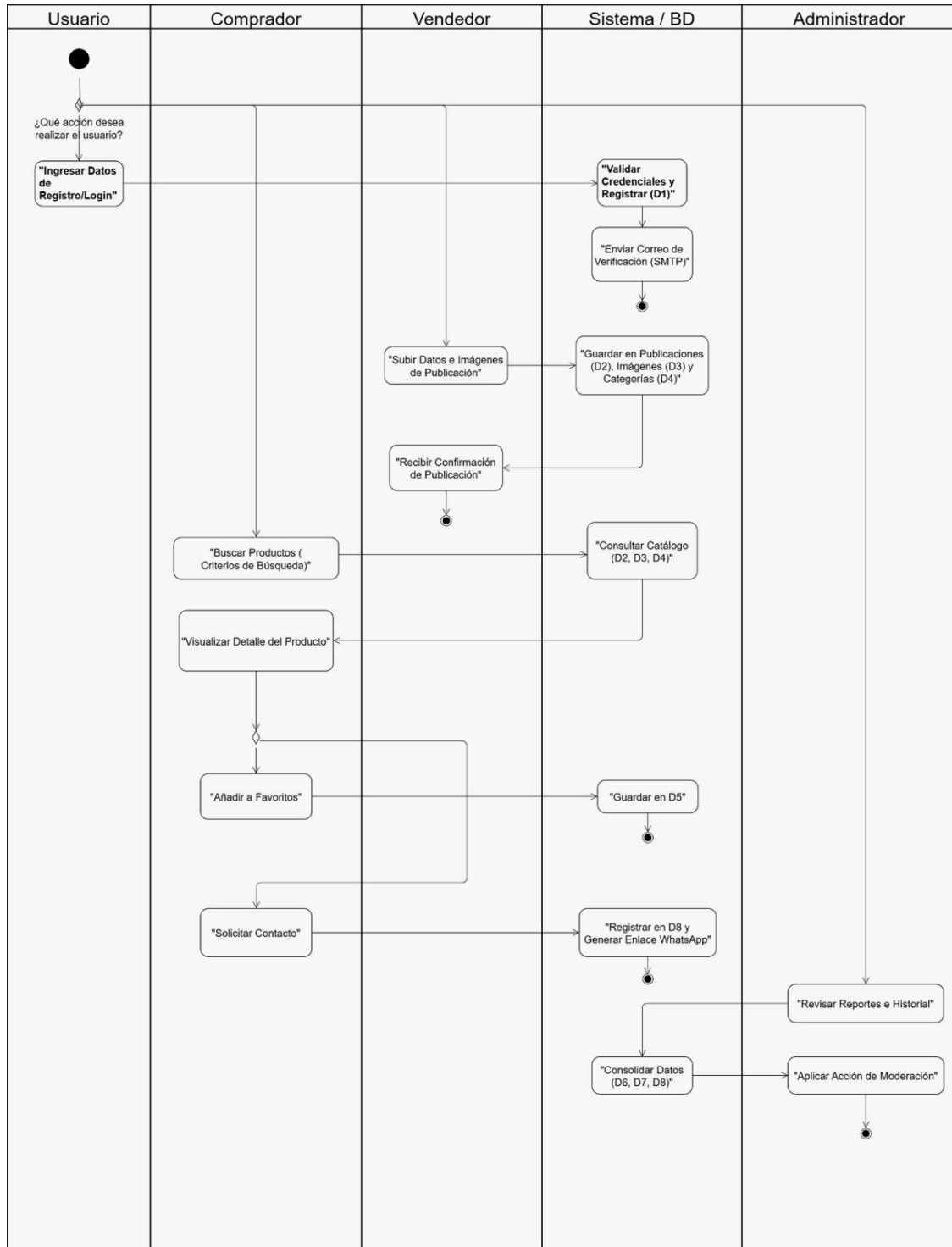
El DFD esencial de UTEQMarket representa el funcionamiento lógico del sistema sin centrarse en tecnologías específicas de implementación. El diagrama organiza el sistema alrededor del actor principal “Miembro de la comunidad UTEQ”, quien interactúa con los procesos esenciales de Registrar y validar miembro, Administrar oferta, Buscar ofertas, Facilitar contacto, Gestionar confianza y Controlar contenido. Cada proceso transforma información relevante del negocio: el registro gestiona los datos personales, de identificación y contacto; la administración de ofertas recibe información del producto o servicio, precio, ubicación y evidencias; la búsqueda utiliza criterios como categoría, rango de precio y ubicación; y el contacto conecta al miembro interesado con el responsable de la oferta mediante un servicio externo de mensajería o correo. Los almacenes de datos Miembros, Ofertas, Evidencias multimedia, Interacciones, Evaluaciones e Incidencias conservan la información necesaria para mantener la continuidad y trazabilidad de las operaciones.

El diagrama también muestra cómo se construye la confianza y se controla el contenido dentro de la plataforma. El proceso Gestionar confianza registra evaluaciones,

comentarios, favoritos e historial de interacciones para calcular o consultar la reputación del vendedor, mientras que Controlar contenido recibe reportes, consulta incidencias y coordina con el Moderador las acciones de revisión y moderación. La relación entre los procesos evidencia que una oferta no se limita a publicarse, sino que puede ser buscada, contactada, evaluada y reportada, generando información que retroalimenta al sistema. En conjunto, este DFD esencial ofrece una visión clara de las funciones de negocio de UTEQMarket y de cómo fluye la información entre usuarios, procesos, almacenes y actores externos, sirviendo como base para validar los requisitos funcionales sin depender de una arquitectura o tecnología concreta.

3.5.4. Diagrama de actividades

Ilustración 6. Diagrama de actividades



Fuente: *visual-paradigm.com*

El Diagrama de Actividades de UTEqMarket representa el flujo de trabajo de las principales operaciones que realizan los diferentes actores dentro de la plataforma, organizando las actividades mediante swimlanes que separan las responsabilidades del

Usuario, Comprador, Vendedor, Sistema/Base de Datos y Administrador. El proceso inicia cuando el usuario decide registrarse o iniciar sesión, ingresando sus credenciales para que el sistema las valide y registre la información en la base de datos de usuarios (D1). Posteriormente, el sistema envía un correo de verificación mediante el servicio SMTP, garantizando que únicamente las cuentas institucionales verificadas puedan acceder a la plataforma. Una vez autenticado, el vendedor puede crear una publicación proporcionando la información del producto y sus imágenes, las cuales son almacenadas en las bases de datos correspondientes (Publicaciones, Imágenes y Categorías), tras lo cual el sistema devuelve una confirmación indicando que la publicación fue registrada correctamente.

Posteriormente, el comprador puede consultar el catálogo utilizando diferentes criterios de búsqueda, visualizar el detalle de los productos, agregarlos a su lista de favoritos o solicitar contacto con el vendedor. Estas acciones generan nuevas operaciones dentro del sistema, como el almacenamiento de favoritos, el registro del historial de contactos y la generación automática del enlace hacia WhatsApp para facilitar la comunicación entre ambas partes. Finalmente, el diagrama incorpora el proceso de moderación, donde el sistema consolida la información proveniente de Reportes, Reseñas e Historial de Contactos para que el Administrador revise las incidencias y aplique las acciones de moderación correspondientes. En conjunto, el diagrama describe de manera secuencial y organizada el comportamiento operativo de UTEQMarket, evidenciando la interacción entre los actores, las actividades del sistema y los principales procesos que soportan el funcionamiento de la plataforma.

3.6. Modelos de comportamiento

3.6.1. Selección del caso

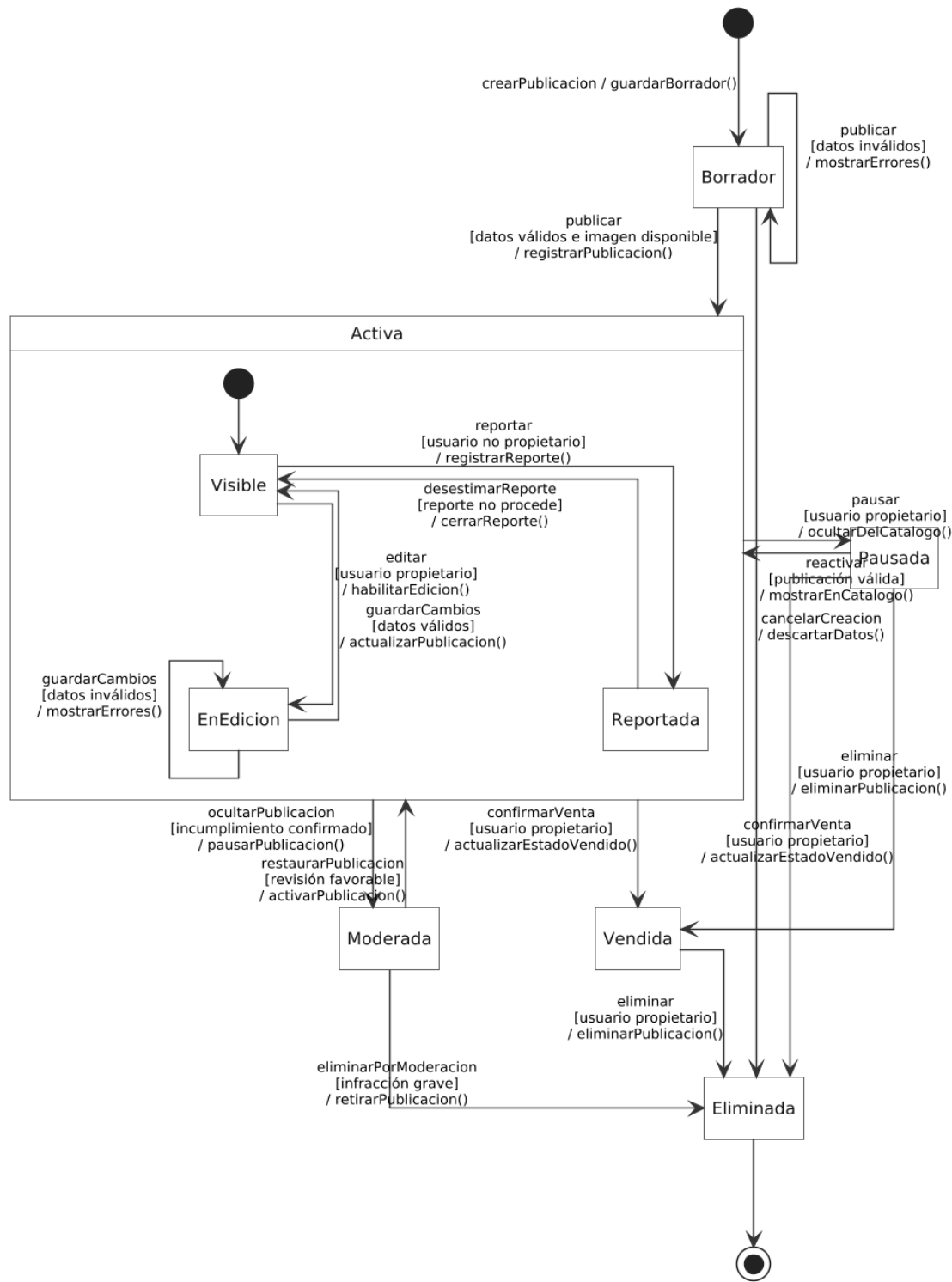
Para elaborar el diagrama de máquina de estados se seleccionó la entidad Publicación, debido a que presenta uno de los ciclos de vida más importantes dentro de UTEQMarket. Esta entidad cambia de condición como resultado de las acciones realizadas por el vendedor y el administrador, por lo que su comportamiento no puede representarse únicamente mediante atributos o relaciones estáticas. La publicación pasa por diferentes etapas desde que es creada hasta que deja de estar disponible en el catálogo, y cada estado determina las operaciones que pueden ejecutarse sobre ella.

La selección también mantiene coherencia con los requisitos funcionales RF-027 Estado inicial, RF-029 Edición de publicación, RF-030 Cambio de estado, RF-031 Eliminación de publicación y RF-059 Ocultamiento de publicación. De acuerdo con estos requisitos, una publicación puede encontrarse activa, pausada o vendida, además de atravesar situaciones internas de edición, revisión o moderación. Por ello, modelar esta entidad permite representar claramente los

eventos, condiciones y acciones que controlan su disponibilidad dentro de la plataforma.

3.6.2. Diagrama de estados

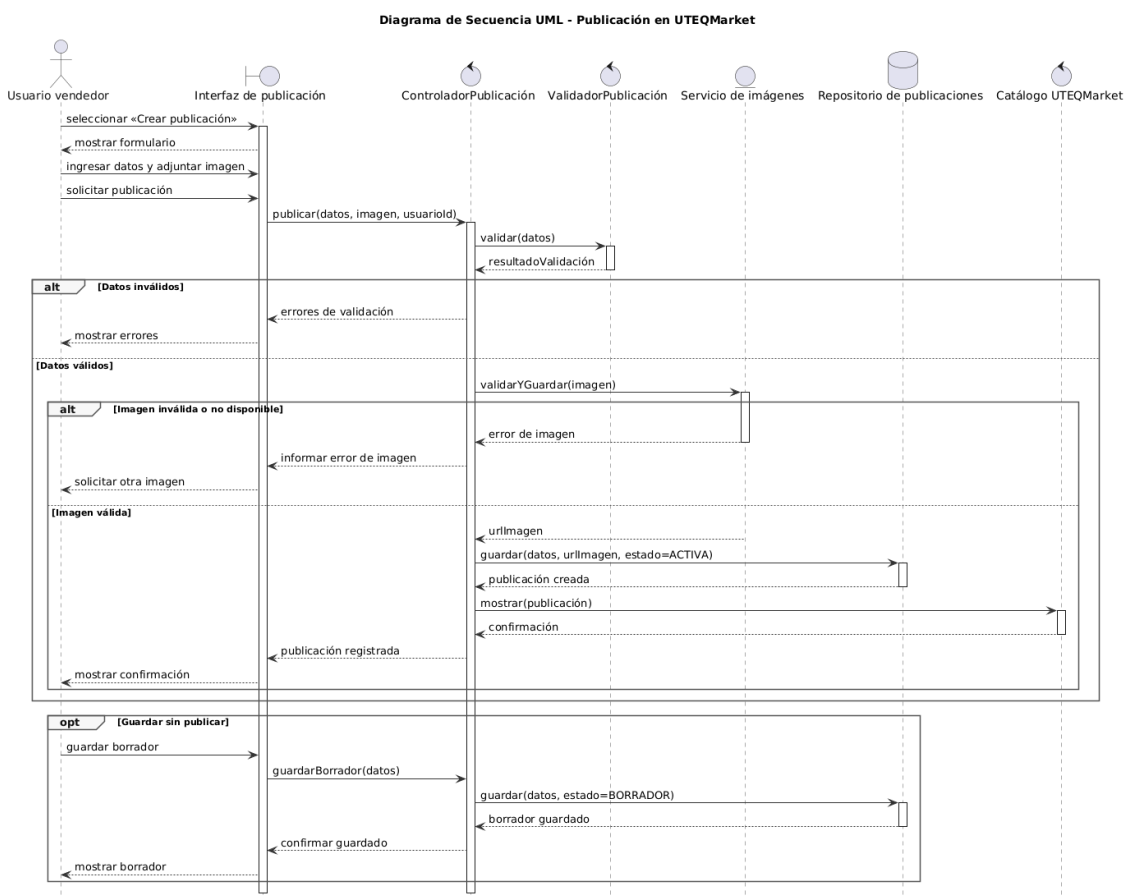
Máquina de Estados UML - Publicación en UTEQMarket



El diagrama de estados representa el ciclo de vida de una publicación dentro de UTEqMarket. El proceso comienza cuando el vendedor crea una nueva oferta, la cual se mantiene en estado Borrador mientras se ingresan y validan los datos requeridos. Cuando la información es válida, la publicación pasa al estado compuesto Activa, dentro del cual puede permanecer visible en el catálogo, entrar temporalmente en edición o cambiar al subestado reportada cuando otro usuario registra una incidencia. Desde el estado activo, el propietario también puede pausarla o marcarla como vendida.

Las transiciones están controladas por eventos, condiciones y acciones específicas. Por ejemplo, una publicación solo puede activarse cuando contiene datos válidos, solo puede ser modificada por su propietario y únicamente el administrador puede ocultarla como resultado de una acción de moderación. Los estados Vendida, Eliminada y Moderada representan posibles resultados finales o restrictivos del ciclo. De esta forma, el modelo evidencia cómo las reglas de negocio y los requisitos de gestión de publicaciones determinan el comportamiento dinámico de esta entidad.

3.6.3. Diagrama de secuencia



El diagrama representa la interacción entre el usuario vendedor y los componentes de UTEQMarket durante la creación de una publicación. El proceso comienza cuando el usuario abre el formulario, ingresa la información del producto, adjunta una imagen y solicita la publicación. La interfaz envía los datos al controlador, que utiliza un componente especializado para validarlos. El fragmento alt muestra los posibles resultados: si existen datos incorrectos, la interfaz presenta los errores; si son válidos, el sistema continúa con la validación y almacenamiento de la imagen.

Cuando tanto los datos como la imagen son válidos, el controlador registra la publicación con el estado ACTIVA y solicita al catálogo que la muestre. Después, el sistema confirma la operación al usuario. El segundo bloque alternativo representa el error ocasionado por una imagen inválida o no disponible, mientras que el fragmento opt incorpora el comportamiento opcional de guardar la información como BORRADOR sin publicarla. De esta manera, el diagrama contempla el flujo exitoso y las principales excepciones relacionadas con la publicación.

3.7. Matriz de trazabilidad

La matriz de trazabilidad permite relacionar cada requisito funcional con los modelos en los que se encuentra representado. Su propósito es comprobar que las funcionalidades especificadas en el SRS hayan sido consideradas durante el análisis del sistema y detectar requisitos sin cobertura, elementos modelados sin respaldo documental o inconsistencias entre diagramas.

El SRS de UTEQMarket contiene 59 requisitos funcionales, por lo que cubrir al menos el 90 % implica relacionar como mínimo 54 requisitos. Para asegurar el cumplimiento completo de la rúbrica, se recomienda incluir los 59 requisitos funcionales.

3.7.1. Matriz RF x Modelos

Leyenda

- **ER:** Modelo Entidad–Relación.
- **DC:** Diagrama de Clases.
- **D0:** DFD Nivel 0.
- **D1:** DFD Nivel 1.
- **DE:** DFD Esencial.
- **DA:** Diagrama de Actividades.
- **ME:** Máquina de Estados.

- **DS:** Diagrama de Secuencia.
- **✓:** El requisito está representado.
- **P:** Representación parcial.
- **—:** No corresponde o no está representado.
- **GAP:** Falta cobertura suficiente.

Registro, autenticación y autorización

Requisito	ER	DC	D0	D1	DE	DA	ME	DS
RF-001 Registro de cuenta	✓	✓	✓	✓	✓	✓	—	—
RF-002 Restricción de dominio institucional	P	✓	✓	✓	✓	✓	—	—
RF-003 Unicidad del correo	✓	✓	—	✓	P	✓	—	—
RF-004 Rechazo de correos desechables	—	P	—	P	—	P	—	—
RF-005 Validación de contraseña	—	✓	✓	✓	P	✓	—	—
RF-006 Confirmación de contraseña	—	P	—	✓	—	✓	—	—
RF-007 Verificación de correo	P	✓	✓	✓	✓	✓	—	—
RF-008 Vigencia del token	—	P	—	P	—	—	—	—

RF-009 Activación de cuenta	✓	✓	✓	✓	✓	✓	—	—
RF-010 Inicio de sesión	✓	✓	✓	✓	P	✓	—	—
RF-011 Cierre de sesión	—	✓	✓	✓	—	P	—	—
RF-012 Control de acceso	P	✓	✓	✓	P	✓	—	—

Catálogo, búsqueda y consulta

Requisito	ER	DC	D0	D1	DE	DA	ME	DS
RF-013 Consulta pública del catálogo	✓	✓	✓	✓	✓	✓	—	—
RF-014 Información resumida	✓	✓	✓	✓	✓	P	—	—
RF-015 Búsqueda textual	✓	✓	✓	✓	✓	✓	—	—
RF-016 Filtrado del catálogo	✓	✓	✓	✓	✓	✓	—	—

RF-017 Ordenamiento de resultados	✓	✓	P	✓	P	P	—	—
RF-018 Persistencia de criterios	—	—	—	GAP	—	—	—	—
RF-019 Detalle de publicación	✓	✓	✓	✓	✓	✓	—	—
RF-020 Perfil público del vendedor	✓	✓	P	P	P	—	—	—

Gestión de publicaciones

Requisito	ER	DC	D0	D1	DE	DA	ME	DS
RF-021 Creación de publicación	✓	✓	✓	✓	✓	✓	✓	✓
RF-022 Asociación de propietario	✓	✓	P	✓	✓	✓	P	✓
RF-023 Validación de contenido	✓	✓	P	✓	P	✓	P	✓
RF-024 Validación de precio	✓	✓	P	✓	P	✓	P	✓

RF-025 Validación de WhatsApp	✓	✓	✓	✓	✓	✓	—	✓
RF-026 Datos de contacto reutilizables	✓	✓	P	✓	P	P	—	P
RF-027 Estado inicial de publicación	✓	✓	P	✓	✓	✓	✓	✓
RF-028 Consulta de publicaciones propias	✓	✓	✓	✓	✓	✓	P	—
RF-029 Edición de publicación	✓	✓	✓	✓	✓	✓	✓	—
RF-030 Cambio de estado	✓	✓	✓	✓	✓	✓	✓	—
RF-031 Eliminación de publicación	✓	✓	✓	✓	✓	✓	✓	—

Gestión de imágenes

Requisito	ER	DC	D0	D1	DE	DA	ME	DS
-----------	----	----	----	----	----	----	----	----

RF-032 Carga de imágenes	✓	✓	✓	✓	✓	✓	—	✓
RF-033 Formatos permitidos	✓	✓	—	✓	P	✓	—	✓
RF-034 Tamaño máximo	✓	✓	—	✓	P	✓	—	✓
RF-035 Almacenamiento seguro	✓	✓	—	✓	✓	P	—	✓
RF-036 Imagen de portada	✓	✓	P	✓	✓	✓	—	✓
RF-037 Adición de imágenes	✓	✓	P	✓	✓	✓	—	✓
RF-038 Eliminación de imágenes	✓	✓	P	✓	✓	✓	—	P
RF-039 Sustitución de portada	✓	✓	—	✓	P	P	—	P

Favoritos y contacto

Requisito	ER	DC	D0	D1	DE	DA	ME	DS
RF-040 Gestión de favoritos	✓	✓	✓	✓	✓	✓	—	—

RF-041 Unicidad de favoritos	✓	✓	—	✓	P	P	—	—
RF-042 Consulta de favoritos	✓	✓	✓	✓	✓	✓	—	—
RF-043 Contacto con vendedor	✓	✓	✓	✓	✓	✓	—	—
RF-044 Registro de contacto	✓	✓	P	✓	✓	✓	—	—
RF-045 Restricción de autocontacto	✓	✓	—	✓	P	P	—	—

Calificaciones y reputación

Requisito	ER	DC	D0	D1	DE	DA	ME	DS
RF-046 Habilitación de calificación	✓	✓	✓	✓	✓	✓	P	—
RF-047 Contenido	✓	✓	P	✓	✓	✓	—	—

de la calificación								
RF-048 Unicidad de calificación	✓	✓	—	✓	P	P	—	—
RF-049 Reputación del vendedor	✓	✓	✓	✓	✓	✓	—	—
RF-050 Presentación de reputación	✓	✓	✓	✓	✓	P	—	—

Reportes y moderación

Requisito	ER	DC	D0	D1	DE	DA	ME	DS
RF-051 Reporte de publicación	✓	✓	✓	✓	✓	✓	✓	—
RF-052 Descripción del reporte	✓	✓	P	✓	✓	✓	—	—
RF-053 Restricciones de reporte	✓	✓	—	✓	P	P	P	—
RF-054 Estado inicial del reporte	✓	✓	P	✓	✓	✓	P	—

RF-055 Panel administrativo	✓	✓	✓	✓	✓	✓	—	—
RF-056 Bandeja de reportes	✓	✓	✓	✓	✓	✓	—	—
RF-057 Revisión de reporte	✓	✓	✓	✓	✓	✓	✓	—
RF-058 Desestimación de reporte	✓	✓	✓	✓	✓	✓	✓	—
RF-059 Ocultamiento de publicación	✓	✓	✓	✓	✓	✓	✓	—

3.7.2. Detección de GAPs

Un gap es una necesidad especificada en los requisitos que no aparece representada suficientemente en los modelos del sistema. Su detección permite identificar funcionalidades que podrían ser omitidas durante el diseño o la implementación.

Durante la verificación de la matriz se detectó un gap relacionado con el RF-018: Persistencia de criterios. Este requisito establece que, después de ejecutar acciones desde el catálogo, UTEQMarket debe conservar la búsqueda, los filtros y el orden seleccionados. Sin embargo, este comportamiento no aparece de forma explícita en el modelo ER, el diagrama de clases, los DFD, el diagrama de actividades ni los modelos de comportamiento. Los diagramas muestran la búsqueda y el filtrado, pero no representan el almacenamiento temporal o la recuperación de los criterios seleccionados.

Para corregir este gap se recomienda incorporar en el diagrama de clases una clase o componente denominado CriterioBúsqueda o EstadoCatalogo, con atributos como texto, categoría, precio mínimo, precio máximo, ubicación y orden. También debe agregarse al diagrama de actividades una acción denominada Conservar criterios seleccionados y otra denominada Restaurar estado del catálogo. Si los criterios se almacenan temporalmente en sesión o en el navegador, esto debe documentarse como una decisión de diseño y no necesariamente como una entidad permanente de la base de datos.

3.7.3. Identificación de gold plating

El gold plating ocurre cuando se agrega al diseño o al software una funcionalidad que no fue solicitada ni aprobada en los requisitos. Aunque pueda parecer una mejora, introduce trabajo adicional, aumenta la complejidad y puede generar inconsistencias con el alcance del proyecto.

En la máquina de estados propuesta se identificó como posible gold plating el estado Borrador. El requisito RF-027 especifica que toda nueva publicación debe registrarse inicialmente como activa, pero el diagrama establece que la publicación comienza en estado Borrador antes de ser activada. El SRS no contiene un requisito que permita guardar publicaciones incompletas para continuarlas posteriormente, por lo que este comportamiento no tiene respaldo documental.

3.7.4. Verificaciones cruzadas entre modelos

Verificación 1: Modelo ER ↔ Diagrama de Clases

Se compararon las entidades del modelo ER con las clases del diagrama de clases. Se comprobó correspondencia entre **Usuario**, **Publicación**, **Categoría**,

ImagenPublicación, Favorito, Contacto, Calificación y Reporte. Las relaciones del ER también aparecen en el diagrama de clases mediante asociaciones y cardinalidades equivalentes.

Por ejemplo:

- Usuario 1 — N Publicación
- Categoría 1 — N Publicación
- Publicación 1 — N ImagenPublicación
- Usuario N — M Publicación, resuelta mediante las clases asociativas Favorito, Contacto, Calificación y Reporte.

La verificación muestra coherencia estructural entre ambos modelos. No obstante, debe revisarse la representación del token de verificación, ya que el SRS exige generar, almacenar y validar un token, pero este elemento no se observa claramente como entidad o atributo del modelo.

Verificación 2: Diagrama de Clases ↔ DFD Nivel 1

Se verificó que los almacenes del DFD Nivel 1 correspondan con las clases y entidades que administran información persistente. Los almacenes **Usuarios, Publicaciones, Imágenes, Categorías, Favoritos, Reportes, Reseñas y Contactos** tienen correspondencia directa con las clases **Usuario, Publicación, ImagenPublicación, Categoría, Favorito, Reporte, Calificación y Contacto**.

Además, los procesos del DFD utilizan los almacenes relacionados con su responsabilidad:

- Gestionar usuarios utiliza Usuarios.
- Gestionar publicaciones utiliza Publicaciones, Imágenes y Categorías.
- Gestionar favoritos utiliza Favoritos.
- Gestionar contacto utiliza Contactos.
- Gestionar reseñas utiliza Calificaciones.
- Gestionar reportes utiliza Reportes y Publicaciones.

Esta correspondencia confirma que los procesos funcionales tienen respaldo en la estructura de datos del sistema.

Verificación 3: Máquina de Estados ↔ Diagrama de Actividades

La máquina de estados de Publicación fue comparada con las actividades relacionadas con la creación, modificación, venta y moderación de ofertas. Se

comprobó que acciones como **crear publicación, validar datos, publicar, pausar, reactivar, marcar como vendida, reportar y moderar** provocan cambios en el estado de la entidad Publicación.

Sin embargo, se detectó una inconsistencia: el diagrama de actividades y el requisito RF-027 registran directamente la publicación como activa después de validar y guardar la información, mientras que la máquina de estados incluye previamente el estado Borrador. Esta diferencia debe corregirse eliminando Borrador o incorporando un requisito que autorice guardar publicaciones incompletas.

Verificación adicional: DFD ⇔ Diagrama de Actividades

El DFD Nivel 1 y el diagrama de actividades representan los mismos procesos principales: registro de usuarios, gestión de publicaciones, consulta del catálogo, favoritos, contacto, calificaciones y moderación. El DFD muestra el movimiento de la información entre actores, procesos y almacenes, mientras que el diagrama de actividades presenta el orden de ejecución y las responsabilidades de cada participante.

Esta comparación confirma que ambos modelos son complementarios: el DFD responde **qué información circula**, mientras que el diagrama de actividades explica **en qué orden se realizan las acciones y quién es responsable de ejecutarlas**.

3.8. Tabla comparativa de modelos

Criterio	Modelo Entidad – Relación (ER)	Diagrama de Clases	DFD	Diagrama de Actividades	Diagrama de Estados	Diagrama de Secuencia
Objetivo	Modelar la estructura de la base de datos y las relaciones entre entidades.	Representar las clases del sistema, sus atributos, operaciones y relaciones orientadas a objetos.	Describir el flujo de información entre procesos, actores y almacenes de datos.	Representar el flujo de actividades y decisiones que conforman un proceso.	Modelar el ciclo de vida de un objeto y sus cambios de estado.	Mostrar el intercambio de mensajes entre los objetos durante la ejecución de un caso de uso.

Perspectiva	Datos.	Datos y diseño orientado a objetos.	Funcional.	Funcional.	Comportamiento.	Comportamiento e interacción.
Notación	Entidades, atributos, relaciones, cardinalidades y llaves.	Clases, atributos, métodos, asociaciones, herencia y multiplicidades.	Procesos, flujos de datos, entidades externas y almacenes de datos.	Actividades, decisiones, swimlanes, fork, join y flujo de objetos.	Estados, transiciones, eventos, acciones y estados compuestos.	Actores, líneas de vida, mensajes, fragmentos (alt, loop, opt) y activaciones.
Ventajas	Reduce redundancia, facilita el diseño de la base de datos y garantiza la integridad de la información.	Facilita la implementación orientada a objetos y la reutilización del código.	Permite comprender cómo circula la información dentro del sistema.	Describe claramente la lógica de los procesos y las responsabilidades de cada actor.	Permite comprender el comportamiento dinámico y el ciclo de vida de una entidad.	Muestra con precisión el orden temporal de las interacciones entre los componentes del sistema.

4. Conclusiones

La aplicación del estándar IEEE/ISO/IEC 29148 permitió estructurar de manera organizada el Documento de Especificación de Requisitos de Software (ERS/SRS), estableciendo una base sólida para el desarrollo de UTEQMarket. El uso de este estándar facilitó la definición clara de los requisitos, mejoró la comunicación entre los interesados y proporcionó una guía para mantener la consistencia y trazabilidad de la información durante el proceso de análisis y diseño.

El proceso de revisión y refinamiento del SRS permitió identificar defectos relacionados con ambigüedades, requisitos incompletos y elementos difíciles de verificar. La aplicación de la metodología EARS (Easy Approach to Requirements Syntax) mejoró la redacción de los requisitos funcionales y no funcionales,

logrando especificaciones más claras, consistentes y medibles, lo que facilita su validación y posterior implementación.

El desarrollo del Modelo Entidad–Relación y del Diagrama de Clases permitió representar adecuadamente la estructura de información de UTEQMarket, definiendo las entidades, atributos, relaciones y cardinalidades necesarias para el funcionamiento del sistema. Asimismo, la verificación de la Tercera Forma Normal (3FN) confirmó que el modelo reduce la redundancia de datos y favorece la integridad y mantenibilidad de la base de datos.

La elaboración de los Diagramas de Flujo de Datos, el Diagrama de Actividades, la Máquina de Estados y el Diagrama de Secuencia permitió representar el sistema desde diferentes perspectivas, describiendo tanto el flujo de información como el comportamiento dinámico de las principales funcionalidades. Estos modelos facilitaron la comprensión de los procesos internos de UTEQMarket y proporcionaron una visión más completa del funcionamiento esperado del sistema antes de su implementación.

La construcción de la matriz de trazabilidad y la comparación entre los diferentes modelos permitieron comprobar que los requisitos funcionales definidos en el SRS se encuentran representados en los distintos diagramas desarrollados. Además, la identificación de brechas (gaps) y casos de gold plating evidenció la importancia de mantener la coherencia entre la especificación de requisitos y los modelos de análisis, contribuyendo a mejorar la calidad del diseño y reduciendo posibles inconsistencias durante el desarrollo del proyecto.

5. Bibliografía

- [1] B. Kitchenham, L. Madeyski, and D. Budgen, “SEGRESS: Software Engineering Guidelines for REporting Secondary Studies,” *IEEE Transactions on Software Engineering*, vol. 49, pp. 1273–1298, 2023, doi: 10.1109/tse.2022.3174092.
- [2] L. Montgomery, D. Fucci, A. Bouraffa, L. Scholz, and W. Maalej, “Empirical research on requirements quality: a systematic mapping study,” *Requir. Eng.*, vol. 27, pp. 183–209, 2022, doi: 10.1007/s00766-021-00367-z.
- [3] D. A. Tamburri, F. Palomba, and R. Kazman, “Success and Failure in Software Engineering: A Followup Systematic Literature Review,” *IEEE Trans. Eng. Manag.*, vol. 68, no. 2, pp. 599–611, 2021, doi: 10.1109/TEM.2020.2976642.
- [4] A. Avignone, A. Tierno, A. Fiori, and S. Chiusano, “Exploring Large Language Models’ Ability to Describe Entity-Relationship Schema-Based Conceptual Data Models,” *Inf.*, vol. 16, p. 368, 2025, doi: 10.3390/info16050368.

- [5] F. Bozyiğit, Ö. Aktaş, and D. Kılınc, "Linking software requirements and conceptual models: A systematic literature review," *Engineering Science and Technology, an International Journal*, vol. 24, no. 1, pp. 71–82, 2021, doi: <https://doi.org/10.1016/j.jestch.2020.11.006>.
- [6] S. Schneider, N. D. Ferreyra, P.-J. Quéval, G. Simhandl, U. Zdun, and R. Scandariato, "How Dataflow Diagrams Impact Software Security Analysis: an Empirical Experiment," in *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2024, pp. 952–963. doi: 10.1109/saner60148.2024.00103.
- [7] S. Lubos *et al.*, "Leveraging LLMs for the Quality Assurance of Software Requirements," in *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, 2024, pp. 389–397. doi: 10.1109/RE59067.2024.00046.
- [8] A. Chaipunyathat and N. Bhumpenpein, "Communication, culture, competency, and stakeholder that contribute to requirement elicitation effectiveness," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 12, p. 6472, Dec. 2022, doi: 10.11591/ijece.v12i6.pp6472-6485.
- [9] X. Franch, M. Glinz, D. Méndez, and N. Seyff, "A Study About the Knowledge and Use of Requirements Engineering Standards in Industry," *IEEE Transactions on Software Engineering*, vol. 48, pp. 3310–3325, 2021, doi: 10.1109/tse.2021.3087792.
- [10] R. Delima, Azhari, and K. Mustofa, "Requirements Engineering Quality: a Literature Review," *Jurnal Nasional Pendidikan Teknik Informatika (JANAPATI)*, p., 2024, doi: 10.23887/janapati.v13i2.53366.
- [11] P. N, "Trends and Challenges in Modern Software Engineering: A Comprehensive Review," *International Journal For Multidisciplinary Research*, p., 2025, doi: 10.36948/ijfmr.2025.v07i03.49852.
- [12] G. Culot, G. Nassimbeni, M. Podrecca, and M. Sartor, "The ISO/IEC 27001 information security management standard: literature review and theory-based research agenda," *The TQM Journal*, p., 2021, doi: 10.1108/tqm-09-2020-0202.
- [13] D. Dori, "Model-based standards authoring: ISO 15288 as a case in point," *Systems Engineering*, vol. 27, pp. 302–314, 2023, doi: 10.1002/sys.21721.
- [14] "IEEE/ISO/IEC International Standard for Software and systems engineering--Software testing--Part 3:Test documentation," *ISO/IEC/IEEE 29119-3:2021(E)*, pp. 1–98, 2021, doi: 10.1109/ieeestd.2021.9591577.

- [15] E. Stephen and E. Mit, "Evaluation of Software Requirement Specification Based on IEEE 830 Quality Properties," *Int. J. Adv. Sci. Eng. Inf. Technol.*, p., 2020, doi: 10.18517/ijaseit.10.4.10186.
- [16] M. Shi, "Documenting Software Requirements Specification: A Revisit," *Comput. Inf. Sci.*, vol. 3, pp. 17–19, 2010, doi: 10.5539/cis.v3n1p17.
- [17] M. Duggal, N. Saxena, and M. Gurve, "SRS Automator - An Attempt to Simplify Software Development Lifecycle," in *2020 6th International Conference on Signal Processing and Communication (ICSC)*, 2020, pp. 278–283. doi: 10.1109/icsc48311.2020.9182768.
- [18] H. Soares and R. Moura, "A methodology to guide writing Software Requirements Specification document," in *2015 Latin American Computing Conference (CLEI)*, 2015, pp. 1–11. doi: 10.1109/clei.2015.7360001.
- [19] M. Broy, "Rethinking Functional Requirements: A Novel Approach Categorizing System and Software Requirements," *Software Technology: 10 Years of Innovation in IEEE Computer*, p., 2018, doi: 10.1002/9781119174240.ch9.
- [20] A. Jarzębowicz and P. Weichbroth, "A Qualitative Study on Non-Functional Requirements in Agile Software Development," *IEEE Access*, vol. 9, pp. 40458–40475, 2021, doi: 10.1109/access.2021.3064424.
- [21] C. Dongmo, "A Review of Non-Functional Requirements Analysis Throughout the SDLC," *Comput.*, vol. 13, p. 308, 2024, doi: 10.3390/computers13120308.
- [22] L. Chung and J. C. S. D. P. Leite, "On Non-Functional Requirements in Software Engineering," pp. 363–379, 2009, doi: 10.1007/978-3-642-02463-4_19.
- [23] V. Santander and J. Castro, "Deriving use cases from organizational modeling," in *Proceedings IEEE Joint International Conference on Requirements Engineering*, 2002, pp. 32–39. doi: 10.1109/icre.2002.1048503.
- [24] M. Ražinskas, B. Miliūnas, M. Jurgelaitis, L. Čėponienė, and L. Bisikirskienė, "Transforming Sketches of UML Use Case Diagrams to Models," *IEEE Access*, vol. 12, pp. 185826–185837, 2024, doi: 10.1109/access.2024.3514455.
- [25] A. Amna and G. Poels, "Systematic Literature Mapping of User Story Research," *IEEE Access*, vol. 10, pp. 51723–51746, 2022, doi: 10.1109/access.2022.3173745.

- [26] Y. A. Kustiawan and T. Y. Lim, "User Stories in Requirements Elicitation: A Systematic Literature Review," in *2023 IEEE 8th International Conference On Software Engineering and Computer Systems (ICSECS)*, 2023, pp. 211–216. doi: 10.1109/icsecs58457.2023.10256364.
- [27] T. Sporseem, T. Dingsøyr, and K.-J. Stol, "User stories as boundary objects in agile requirements engineering: A theoretical literature review," *J. Syst. Softw.*, vol. 233, p. 112693, 2025, doi: 10.1016/j.jss.2025.112693.
- [28] P. Skavantzios and S. Link, "Entity/Relationship Graphs: Principled Design, Modeling, and Data Integrity Management of Graph Databases," *Proceedings of the ACM on Management of Data*, vol. 3, pp. 1–26, 2025, doi: 10.1145/3709690.
- [29] R. Wieringa, "Entity-Relationship Diagrams," pp. 77–88, 2003, doi: 10.1016/b978-155860755-2/50013-7.
- [30] S. M. Pulungan, R. Febrianti, T. Lestari, N. Gurning, and N. Fitriana, "Analisis Teknik Entity-Relationship Diagram Dalam Perancangan Database," *Jurnal Ekonomi Manajemen dan Bisnis (JEMB)*, p., 2023, doi: 10.47233/jemb.v1i2.533.
- [31] R. Rashkovits and I. Lavy, "Mapping Common Errors in Entity Relationship Diagram Design of Novice Designers," *International Journal of Database Management Systems*, p., 2021, doi: 10.5121/ijdms.2021.13101.
- [32] S. Tabassam and E. Al-Qahtane, "Comparative Analysis on Requirement Engineering Modelling Techniques Case Study of Personal Al-Haj E-Guide," in *2019 2nd International Conference on Computer Applications & Information Security (ICCAIS)*, 2019, pp. 1–8. doi: 10.1109/cais.2019.8769549.
- [33] N. Selamat, "Similarity Syntax Rules between DFD and UML Diagrams," *International Journal of Advanced Trends in Computer Science and Engineering*, p., 2019, doi: 10.30534/ijatcse/2019/70832019.
- [34] D. Wahyu, Nugraha, A. H. Rustam, and M. Basri, "Peran Data Flow Diagram dalam Mengidentifikasi Kebutuhan Fungsional Sistem Informasi," *Jurnal Sistem Informasi, Sains Data, dan Informatika*, p., 2026, doi: 10.56956/7s2ptx53.
- [35] Q. Li and Y. Chen, "Data Flow Diagram," pp. 85–97, 2009, doi: 10.1007/978-3-540-89556-5_4.
- [36] S. Abrahão, C. Gravino, E. Insfrán, G. Scanniello, and G. Tortora, "Assessing the Effectiveness of Sequence Diagrams in the Comprehension of

Functional Requirements: Results from a Family of Five Experiments,” *IEEE Transactions on Software Engineering*, vol. 39, pp. 327–342, 2013, doi: 10.1109/tse.2012.27.

- [37] T. Ambreen, N. Ikram, M. Usman, and M. Niazi, “Empirical research in requirements engineering: trends and opportunities,” *Requir. Eng.*, vol. 23, pp. 63–95, 2016, doi: 10.1007/s00766-016-0258-2.
- [38] A. Aleryani, “Analyzing Data Flow: A Comparison between Data Flow Diagrams (DFD) and User Case Diagrams (UCD) in Information Systems Development,” *European Modern Studies Journal*, p., 2024, doi: 10.59573/emsj.8(1).2024.28.
- [39] N. Sitlong and F. Nonyelum, “Harmonization of Tools and Techniques for System Development,” vol. 9, p., 2020, doi: 10.31871/wjir.9.1.28.
- [40] M. Chonoles, “Behavior: State Machine Diagrams,” pp. 313–342, 2017, doi: 10.1016/b978-0-12-809640-6.00019-2.
- [41] S. Al-Fedaghi, “State-Based Behavior Modeling in Software and Systems Engineering,” *ArXiv*, vol. abs/2205.07018, p., 2022, doi: 10.48550/arxiv.2205.07018.
- [42] S. Uchitel, T. Systä, and A. Zündorf, “Scenarios and state machines: models, algorithms, and tools,” pp. 659–660, 2002, doi: 10.1145/581339.581431.
- [43] S. Al-Fedaghi, “UML Sequence Diagram: An Alternative Model,” *ArXiv*, vol. abs/2105.15152, p., 2021, doi: 10.14569/ijacsa.2021.0120576.
- [44] É. André, S. Liu, Y. Liu, C. Choppy, J. Sun, and J. Dong, “Formalizing UML State Machines for Automated Verification – A Survey,” *ACM Comput. Surv.*, vol. 55, pp. 1–47, 2023, doi: 10.1145/3579821.